YEDITEPE UNIVERSITY
FACULTY OF ENGINEERING

MICROCONTROLLER BASED AI SYSTEM FOR PLANT AND FRUIT HEALTH MONITORING IN TOMATO FARMING

Ahmet Hasan Okumuş

GRADUATION PROJECT REPORT

Department of Electrical and Electronics Engineering

Supervisor

Asst. Prof. Dr. İpek Baz

ISTANBUL, 2026

YEDITEPE UNIVERSITY
FACULTY OF ENGINEERING

Microcontroller Based AI System for Plant and Fruit Health Monitoring in Tomato Farming

by

Ahmet Hasan Okumuş

June 1, 2026, Istanbul

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

YEDITEPE UNIVERSITY

Signature of Author(s)
……

Department of Electrical and Electronics Engineering

Certified By
……

Project Supervisor, Department of Electrical and Electronics Engineering

Accepted By
……

Head of the Department of Electrical and Electronics Engineering

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor, Asst. Prof. Dr. İpek Baz, for her guidance, support, and valuable feedback during this graduation project.

I would also like to thank my friends who helped me during the image collection and dataset preparation process. Their support was especially valuable during the greenhouse image collection stage.

Finally, I would like to thank my family for their continuous support, patience, and encouragement throughout my education.

June, 2026

Ahmet Hasan Okumuş

TABLE OF CONTENTS

ACKNOWLEDGEMENTS 1
ABSTRACT 5
LIST OF SYMBOLS 7
ABBREVIATIONS 8
LIST OF FIGURES 9
LIST OF TABLES 10
1. INTRODUCTION 12
    1.1. Thesis Content 18
2. RESEARCH OBJECTIVE 20
    2.1. Problem Definition 20
    2.2. Main Objective of the Project 21
    2.3. Dataset-Related Objective 22
    2.4. Class Selection Objective 23
    2.5. Model Development Objective 24
    2.6. Embedded Deployment Objective 25
    2.7. Summary of Research Objectives 26
3. LITERATURE REVIEW 28
    3.1. Deep Learning in Plant Disease Detection 28
    3.2. Tomato Leaf Disease Classification 29
    3.3. Lightweight CNN Models for Mobile and Embedded Systems 30
    3.4. High-Capacity CNN Models as Reference Experiments 31
    3.5. Object Detection and Disease Localization Studies 31
    3.6. Research Gap and Relation to This Project 32
4. DESIGN 35
    4.1. Realistic Constraints and Conditions 36
        4.1.1. Economic Constraints 36
        4.1.2. Environmental Constraints 36
        4.1.3. Sustainability 37
        4.1.4. Manufacturability 37
        4.1.5. Ethical Considerations 37
        4.1.6. Health and Safety 38
        4.1.7. Social and Practical Constraints 38
        4.1.8. Technical Constraints 38
    4.2. Cost of the Design 39
    4.3. Engineering Standards 40
    4.4. Details of the Design 43
        4.4.1. Dataset Design 43

4.4.2. Model Design 44
4.4.3. Input Image Design 48
4.4.4. Model Conversion and Quantization Design 48
4.4.5. Embedded System Design 49
4.4.6. Testing Design 50
5. METHODS 51
5.1. Overall Methodology 51
5.2. Dataset Collection 52
5.3. Image Labelling and Dataset Organization 54
5.4. Dataset Cleaning 56
5.5. Class Selection and Final Dataset Split 58
5.6. Image Preprocessing 61
5.7. Data Augmentation 64
5.8. Model Development Experiments 65
5.9. SqueezeNet-Inspired Lightweight CNN Experiments 68
5.10. EfficientNetB3 Upper-Bound Experiments 71
5.11. Final ST FD-MobileNetV1 Model 73
5.12. Model Evaluation in Software Environment 75
5.13. TensorFlow Lite Conversion and Quantization 76
5.14. Test Image Export for STM32 77
5.15. STM32 Deployment 77
5.16. Evaluation Method 78
6. RESULTS AND DISCUSSION 80
6.1. Dataset Summary 80
6.2. Model Development Summary 82
6.3. Preprocessing Experiments 85
6.4. EfficientNetB3 Upper-Bound Results 86
6.5. SqueezeNet-Inspired Lightweight CNN Results 88
6.6. Final ST FD-MobileNetV1 Software Result 90
6.7. Final Per-Class Software Results 91
6.8. Quantization Result 95
6.9. STM32 SD Card Test Result 96
6.10. STM32 Per-Class Results 98
6.11. STM32 Confusion Matrix Discussion 100
6.12. Discussion of Satisfactory and Unsatisfactory Parts 101
7. CONCLUSION 103
REFERENCES 106
APPENDICES 108
Appendix A 108

A.1 General Settings 108
A.2 Dataset Preparation and Class Selection 109
A.3 Train / Validation / Test Split 110
A.4 Final Model Configuration 111
A.5 Preprocessing and Data Augmentation 111
A.6 Training and Quantization Settings 112
A.7 Software Test Evaluation 113
A.8 Classification Report and Confusion Matrix 114
A.9 Final Model Saving 115
Appendix B 116
Appendix C 118
Appendix D 119
D.1 Main Model Experiment Results 119
D.2 Final ST FD-MobileNetV1 Training Settings 122
D.3 Final ST FD-MobileNetV1 Software Results 123
D.4 STM32 AI Analyze Results 125
D.5 STM32 SD Card Test Results 125
D.6 Final Selection Summary 127
Appendix E 127
E.1 Preprocessing and STM32 Binary Export Code 127
E.2 Binary Export Function 128
E.3 Exporting the Test Set 128
E.4 SD Card Folder Structure 129
E.5 Importance of Binary Export 130

ABSTRACT

This project presents a microcontroller-based artificial intelligence system for tomato plant health monitoring. The main aim of the project is to classify tomato leaf images using a lightweight deep learning model and to deploy the trained model on an STM32 microcontroller. For this purpose, a custom tomato plant image dataset was collected from greenhouse conditions over several weeks. The collected images were manually labelled using CVAT and cleaned several times to remove unclear, low-quality, and visually ambiguous samples.

The initial dataset included different plant parts such as leaves, fruits, stems, and flowers. However, unhealthy fruit samples were very limited, and there were no unhealthy stem or flower samples in the collected dataset. Therefore, these categories were not used in the final classifier in order to avoid misleading accuracy values caused by class imbalance. In addition, back-side leaf images were removed from the final dataset because their sample number was lower and their visual appearance was different from front-side leaf images. Since disease and pest symptoms are generally easier to observe on the upper/front surface of the leaf, the final dataset was restricted to front-side leaf images only.

The final classification problem included five front-side leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf. Several model architectures were tested during the project, including EfficientNetB3, SqueezeNet-like lightweight CNN models, and ST FD-MobileNetV1 models from the STM32 AI Model Zoo. EfficientNetB3 was used as a high-capacity software reference model to estimate the upper performance limit of the cleaned dataset. SqueezeNet-like models were tested as custom lightweight embedded candidates.

For the final embedded implementation, ST FD-MobileNetV1 was selected because it provided a good balance between classification accuracy and STM32 deployment suitability. The final ST FD-MobileNetV1 model achieved 91.25% test accuracy in the software environment. After deployment on the STM32F746G board, the model was tested using preprocessed image files stored on an SD card. In the STM32 test, 1405 images were tested and 1276 images were classified correctly, giving an embedded test accuracy of 90.81%. The

results show that tomato leaf health classification can be performed on a microcontroller-based embedded system with a lightweight AI model.

LIST OF SYMBOLS

H: Image height

W: Image width

C: Number of image channels

N: Number of samples

MB: Megabyte

KB: Kilobyte

ABBREVIATIONS

AI: Artificial Intelligence

CNN: Convolutional Neural Network

CVAT: Computer Vision Annotation Tool

DL: Deep Learning

EMA: Exponential Moving Average

F1-score: Harmonic Mean of Precision and Recall

FD-MobileNet: Fast Depthwise MobileNet

Flash: Non-volatile Program Memory

GPU: Graphics Processing Unit

LCD: Liquid Crystal Display

MCU: Microcontroller Unit

RAM: Random Access Memory

RGB: Red, Green, Blue

SD Card: Secure Digital Card

STM32: STMicroelectronics 32-bit Microcontroller Family

TFLite: TensorFlow Lite

UART: Universal Asynchronous Receiver/Transmitter

USB: Universal Serial Bus

X-CUBE-AI: STM32Cube Expansion Package for Artificial Intelligence

LIST OF FIGURES

Figure 1.1 Example of a visually unclear flies leaf image with low symptom visibility 14
Figure 1.2 Example of a visually unclear nutdef leaf image with low symptom visibility 14
Figure 1.3 Example of a clearly visible early blight leaf image from the PlantVillage dataset 15
Figure 1.4 Example of a clearly visible bacterial spot leaf image from the PlantVillage dataset 15
Figure 1.5 Example of a low-visibility leaf image whose label was corrected from trips_leaf to flies_leaf during dataset cleaning 16
Figure 1.6 Example of a low-visibility leaf image whose label was corrected from healthy_leaf to nutdef_leaf during dataset cleaning 17
Figure 4.1 The general design workflow 36
Figure 4.2 Examples of visually similar leaf classes: trips_leaf and flies_leaf 45
Figure 4.3 STM32 Inference Workflow 50
Figure 5.1 Overall methodology workflow 52
Figure 5.2 General view of the greenhouse used for dataset collection 55
Figure 5.3 Example of manual image labelling in CVAT 56
Figure 5.4 Effect of resizing on a flies_leaf sample 59
Figure 5.5 Effect of resizing on a trips_leaf sample 59
Figure 5.6 Example images from the final five front-side leaf classes 61
Figure 5.7 Comparison of original, 224×224, 192×192, and 128×128 versions of the same nutdef_leaf image 65
Figure 5.8 The simplified quantized model evaluation process 78
Figure 5.9 STM32 inference pipeline 79
Figure 6.1 UART output of the final STM32 SD card test 97
Figure 6.2 Confusion matrix for the STM32 test 101

LIST OF TABLES

Table 2.1 Final class labels used in the classification task 23
Table 2.2 Model selection criteria 24
Table 2.3 Embedded deployment and inference steps 26
Table 3.1 Comparison of related studies and this project 34
Table 4.1 Cost of the Design 39
Table 4.2 Engineering standards related to the project 42
Table 4.3 Final selected front-side leaf classes 43
Table 4.4 Main stages of the model design process 47
Table 4.5 Final testing design 50
Table 5.1 Original labels to final classes mapping 56
Table 5.2 The final dataset distribution 61
Table 5.3 Effect of input image size on memory and visual detail 62
Table 5.4 Preprocessing approaches tested during the project 63
Table 5.5 Main model development stages 68
Table 5.6 SqueezeNet-inspired experiment stages 71
Table 5.7 EfficientNetB3 experiment stages 73
Table 5.8 ST FD-MobileNetV1 experiment stages 75
Table 5.9 Configuration of the final ST FD-MobileNetV1 model 76
Table 5.10 Final evaluation stages 80
Table 6.1 Final front-side leaf dataset distribution 82
Table 6.2 Summary of selected model development experiments 84
Table 6.3 Summary of preprocessing and augmentation experiments 86
Table 6.4 Summary of EfficientNetB3 results 88
Table 6.5 Summary of SqueezeNet-inspired CNN results 90
Table 6.6 Summary of ST FD-MobileNetV1 results 91
Table 6.7 Final ST FD-MobileNetV1 software result 92
Table 6.8 Per-class software results of the final ST FD-MobileNetV1 model 93
Table 6.9 Software confusion matrix of the final ST FD-MobileNetV1 model 94
Table 6.10 Software and embedded test accuracy comparison 95

Table 6.11 Final STM32 SD card test result 98
Table 6.12 Per-class STM32 results 99
Table C.1 STM32 AI analyze summary of the final quantized model 119
Table D.1 Numerical summary of main model experiments 120
Table D.2 Final ST FD-MobileNetV1 training configuration 123
Table D.3 Final preprocessing and augmentation settings 124
Table D.4 Final ST FD-MobileNetV1 software result 124
Table D.5 Per-class software results 125
Table D.6 STM32 AI analyze summary 127
Table D.7 Final STM32 SD card test result 127
Table D.8 Per-class STM32 results 128

1. INTRODUCTION

Tomato farming is an important part of agriculture because tomato is widely consumed and economically valuable. However, tomato plants can be affected by different diseases, pests, and nutrient problems during their growth. These problems may reduce crop quality and yield if they are not detected early. In many farms and greenhouses, plant health is still checked mostly by visual inspection. This method depends on human experience and may take a long time when the number of plants is high.

In traditional image processing approaches, plant disease detection is usually based on manually designed features such as color thresholds, edge information, shape, or texture. These methods can be useful in controlled images, but they may be sensitive to lighting changes, background differences, and leaf position. In recent years, computer vision and deep learning methods have been used more widely for plant disease detection[1] . Deep learning models can classify plant images by learning visual patterns such as color changes, spots, texture differences, and damaged regions. Several studies show that convolutional neural networks can reach high accuracy for tomato leaf disease classification [2] ,[3]. However, many successful models are large and need high computational power [4], [5]. This is a problem for small embedded systems, where memory, processing power, and energy are limited.

The aim of this project is to develop a microcontroller-based artificial intelligence system for tomato plant health monitoring. The system is designed to classify tomato leaf images using a lightweight deep learning model and to run the trained model on an STM32 microcontroller. Instead of using a computer or cloud server during inference, the final goal is to perform classification locally on the embedded board.

Tomato leaf health classification is not a simple image classification problem in real greenhouse conditions. The images are affected by different lighting conditions, different camera angles, leaf positions, background objects, and different symptom visibility levels. Some pest symptoms are also very small and may become difficult to observe after resizing the image to the model input size. For example, leaves with only a few visible flies can look similar to healthy leaves after resizing. For this reason, dataset quality and symptom visibility

were important issues throughout the project.



Figure 1.1 Example of a visually unclear flies leaf image with low symptom visibility



Figure 1.2 Example of a visually unclear nutdef leaf image with low symptom visibility

A major difficulty of this project was the creation of the dataset. The images were not taken from a ready and clean public dataset. Instead, tomato plant images were collected from real greenhouse conditions using mobile phone cameras. Every week, different students collected images from the greenhouse. These images were then labelled manually using CVAT. After

labelling, the data was reviewed several times by the project members.



Figure 1.3 Example of a clearly visible early blight leaf image from the PlantVillage dataset



Figure 1.4 Example of a clearly visible bacterial spot leaf image from the PlantVillage dataset

The dataset preparation process included many decisions. Some images were removed because they were unclear, low-resolution, or visually ambiguous. Some pest-related images were also removed when the visible evidence was too weak, for example leaves containing only one very small insect. These images could create unreliable labels, especially after resizing to the model input size. During the project, dataset decisions were discussed with the project supervisor and the academic partners of the project. After these discussions, labelled data was shared among the students for checking, cleaning, and correction before

being added back to the main dataset pool. In addition, during the model training process, the dataset was reviewed again several times. Some images were removed, some images were added, and some class labels were corrected when necessary.

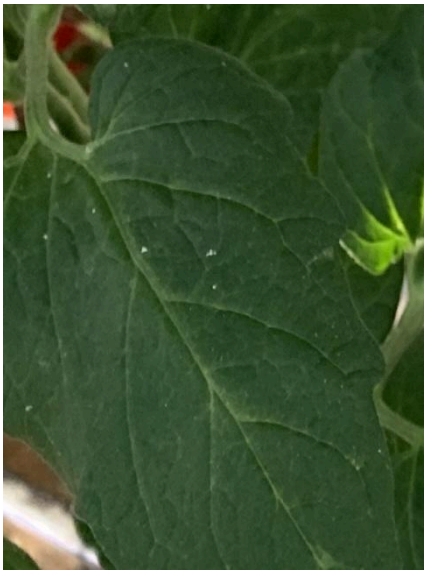


Figure 1.5 Example of a low-visibility leaf image whose label was corrected from trips_leaf to flies_leaf during dataset cleaning



Figure 1.6 Example of a low-visibility leaf image whose label was corrected from healthy_leaf to nutdef_leaf during dataset cleaning

Another important difficulty was finding enough diseased or pest-affected samples. Since the plants were grown in controlled greenhouse conditions, most plants were healthy most of the time. Some disease or pest symptoms appeared only in certain weeks. In some cases, several plants were affected during the same period, but after treatment and greenhouse maintenance, the symptoms decreased in the following days or weeks. Therefore, the dataset naturally changed from week to week, and obtaining balanced class data was difficult.

Some images were also difficult to capture because of physical and environmental limitations. Upper leaves were sometimes hard to reach, and the angle of sunlight made it difficult to take clear images at certain heights. These problems caused variations in lighting, focus, distance, and image quality.

At the beginning of the project, the scope included different tomato plant parts such as leaves, fruits, stems, and flowers. However, the collected dataset was not suitable for a reliable fruit, stem, and flower classification task. Unhealthy fruit samples were very limited, and no unhealthy stem or flower samples were available in the collected dataset. Including these categories could produce misleading accuracy values because the model could mostly learn the dominant healthy classes. For this reason, fruit, stem, and flower categories were not used in the final classifier.

Another important decision was related to front-side and back-side leaf images. Back-side leaf images had fewer samples and their visual appearance was different from front-side leaf images. This increased the visual variation inside the dataset and made the classification problem more confusing. Also, in practical greenhouse inspection, leaves are usually checked from the upper/front side first, and many disease or pest symptoms are easier to observe on the upper surface of the leaf. Therefore, the final dataset was restricted to front-side leaf images only.

The final embedded model was trained for five front-side leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf. The tuta_leaf class had fewer samples than the other classes, but it was kept in the final system because it is an important pest-related problem in

tomato farming. Removing this class would make the classification task easier, but it would reduce the practical value of the system.

Several model architectures were tested during the project, including EfficientNetB3, SqueezeNet-like lightweight CNN models, and STM32 AI Model Zoo based models. EfficientNetB3 was useful to understand the possible performance limit of the cleaned dataset. However, it was not selected as the final embedded model because it was not suitable for the target microcontroller system. SqueezeNet-like models were tested as custom lightweight candidates. For the final embedded implementation, the ST FD-MobileNetV1 model from the STM32 AI Model Zoo was selected because it provided a good balance between classification performance and embedded deployment suitability.

The main challenges of the project can be summarized as follows:

● collecting enough tomato leaf images from real greenhouse conditions,

● manually labelling the collected images using CVAT,

● reviewing and cleaning data collected by different project members,

● removing unclear, low-quality, weakly labelled, and visually ambiguous samples,

● finding enough diseased or pest-affected samples in controlled greenhouse conditions,

● selecting a consistent front-side leaf dataset,

● dealing with class imbalance, especially for the tuta_leaf class,

● preserving small disease and pest details after image resizing,

● testing different preprocessing, augmentation, and model alternatives,

● selecting a model that gives acceptable accuracy and can also be deployed on STM32.

The main contributions of this project are listed below:

● A custom tomato leaf image dataset was created from real greenhouse conditions instead of using only a ready public dataset.

● The image collection process was carried out over several weeks using mobile phone cameras.

● The collected images were manually labelled using CVAT and repeatedly checked by the project members.

● The dataset was cleaned several times by removing unclear, low-resolution, weakly labelled, and visually ambiguous samples.

● Dataset decisions were made through discussion and review, including whether to remove weak pest samples, back-side leaf images, and non-leaf categories.

● The final dataset was restricted to front-side leaf images to obtain a more consistent and practical classification problem.

● The tuta_leaf class was kept in the final dataset although its sample count was low, because it is an important pest-related class in tomato farming.

● Different model and preprocessing alternatives were tested, including input size changes, interpolation methods, augmentation changes, EfficientNetB3, SqueezeNet-like CNNs, and ST FD-MobileNetV1 models.

● EfficientNetB3 was used as a high-capacity software reference model, while SqueezeNet-like and ST FD-MobileNetV1 models were tested as lightweight embedded candidates.

● The final ST FD-MobileNetV1 model was deployed and tested on the STM32F746G board using SD card input images.

1.1. Thesis Content

This report is organized as follows. Chapter 1 introduces the motivation of the project and explains the main problem, project challenges, and contributions. Chapter 2 presents the research objective and the scope of the study in more detail. Chapter 3 gives a literature

review about plant disease detection, tomato leaf classification, lightweight CNN models, and embedded AI systems. Chapter 4 explains the design of the proposed system, including realistic constraints, cost, standards, and design details.

Chapter 5 describes the methods used in the project. This includes dataset collection, CVAT labelling, dataset cleaning, class selection, preprocessing, model training, quantization, and STM32 deployment. It also explains the main model development steps, including the SqueezeNet-like lightweight CNN experiments, EfficientNetB3 upper-bound experiments, and the final ST FD-MobileNetV1 model.

Chapter 6 presents the experimental results and discusses the performance of different models. It includes software-level results, preprocessing experiments, model comparison, final embedded model results, and STM32 SD card test results. Chapter 7 concludes the report and gives possible future improvements.

2. RESEARCH OBJECTIVE

2.1. Problem Definition

The main problem of this study is to develop a tomato leaf health classification system that can work with images collected from real greenhouse conditions and can also be deployed on a microcontroller-based embedded platform. Therefore, the project is not only a standard software classification problem. It is a practical embedded AI problem that includes both real-world dataset difficulties and hardware limitations.

In many public plant disease datasets, images are usually clean, centered, and taken under controlled conditions. However, real greenhouse images are more difficult. They can include different lighting conditions, shadows, camera angles, leaf positions, background objects, and different symptom visibility levels. In addition, some pest symptoms are very small and may become difficult to observe after resizing the image to the model input size. Therefore, creating a reliable dataset from real greenhouse images is one of the main problems of this project.

The second part of the problem is embedded deployment. A deep learning model that performs well on a computer may not be suitable for a microcontroller. The STM32F746G board has limited memory and processing capacity. For this reason, the selected model must be lightweight enough to be converted, quantized, deployed, and tested on the STM32 platform.

In this project, the problem was made more specific by focusing on front-side tomato leaf images. Fruit, stem, and flower categories were not used in the final model because the dataset was not suitable for a reliable classification task for these plant parts. Unhealthy fruit samples were very limited, and no unhealthy stem or flower samples were available in the collected dataset. Back-side leaf images were also removed because their sample number was lower and their visual appearance was different from front-side leaf images.

Therefore, the final problem can be defined as developing a lightweight embedded AI system for classifying front-side tomato leaf health conditions using real greenhouse image data.

2.2. Main Objective of the Project

The main objective of this project is to design and implement a microcontroller-based artificial intelligence system that can classify tomato leaf health conditions using images collected from real greenhouse conditions. The system uses front-side tomato leaf images as input and produces a class prediction by using a lightweight deep learning model deployed on an STM32 microcontroller.

The complete workflow of the project includes:

● reviewing related studies on plant disease detection, tomato leaf classification, lightweight CNN models, and embedded AI,

● collecting tomato plant images from real greenhouse conditions,

● manually labelling the collected images using CVAT,

● checking, cleaning, and correcting the dataset several times,

● selecting a practical and consistent front-side leaf dataset,

● training and comparing different CNN-based models,

● testing preprocessing, image size, interpolation, and augmentation approaches,

● selecting an embedded-compatible model,

● converting and quantizing the selected model,

● testing the selected model on the STM32 platform using SD card input images.

The final goal is to demonstrate that tomato leaf health classification can be performed on a microcontroller instead of only on a computer or cloud system.

2.3. Dataset-Related Objective

One of the most important objectives of this project was to create a reliable dataset. The images were collected manually from tomato plants in greenhouse conditions over several weeks. Different project members collected images using mobile phone cameras. After image collection, the images were labelled using CVAT.

During the first experiments, it was observed that dataset quality strongly affected model performance. Some images were low-resolution, unclear, or visually ambiguous. Some pest-related samples also had very weak visual evidence, such as leaves with only one very small insect. These samples were removed because they could create unreliable labels.

Another important observation was made after resizing some images to common model input sizes such as 224×224 and 192×192 pixels. In some resized images, the disease or pest symptoms became difficult to see even by human observation. Therefore, dataset cleaning was necessary to make the training data more meaningful for the model.

The dataset was not cleaned only once. It was reviewed several times during the project. Labelled data collected by different project members was checked, corrected, and cleaned before being added to the main dataset pool. During model training experiments, some samples were reviewed again, and unclear or wrongly classified images were removed or corrected when necessary.

The objective of this step was not only to increase the number of images, but to obtain a cleaner, more consistent, and more reliable dataset. This dataset preparation process was one of the most important contributions of the project.

2.4. Class Selection Objective

The initial project scope included leaves, fruits, stems, and flowers. However, the dataset was not balanced for all plant parts. Healthy fruit samples were available in large numbers, but unhealthy fruit samples were very limited. In addition, unhealthy stem and flower samples were not available in the collected dataset.

Including these categories in the final model could produce misleading accuracy values. The model could learn the dominant healthy classes and still reach a high overall accuracy, although it would not have enough data to learn unhealthy fruit, stem, or flower conditions properly. For this reason, fruit, stem, and flower categories were excluded from the final classifier.

Another class selection decision was related to front-side and back-side leaf images. Back-side leaf images had fewer samples compared with front-side leaf images. Their color, texture, and visual appearance were also different from the upper/front side of the leaf. This difference increased the variation inside the dataset and made the classification problem more confusing.

In practical greenhouse inspection, leaves are usually checked from the upper/front side first. Disease and pest symptoms are also generally easier to observe on the upper surface of the leaf. Therefore, the final classification task was restricted to front-side leaf images only.

Table 2.1 Final class labels used in the classification task

Class No	Class Name
1	flies_leaf
2	healthy_leaf
3	nutdef_leaf
4	trips_leaf
5	tuta_leaf

The tuta_leaf class had fewer samples than the other classes. However, it was kept in the final dataset because it is an important pest-related problem in tomato farming. Removing this class would make the classification task easier, but it would reduce the practical value of the system. Therefore, tuta_leaf was included in the final model despite its lower sample count.

2.5. Model Development Objective

Another objective was to compare different CNN-based models and select a model that is suitable for embedded deployment. During the project, several architectures were tested, including EfficientNetB3, SqueezeNet-like lightweight CNN models, and ST FD-MobileNetV1 models from the STM32 AI Model Zoo.

The model selection was based on three main criteria:

Table 2.2 Model selection criteria

Criterion	Purpose
Classification accuracy	The model should classify leaf images correctly.
Model size	The model should fit embedded system limitations.
Deployment feasibility	The model should be convertible and usable on STM32.

Larger models such as EfficientNetB3 were useful to understand the possible performance limit of the cleaned dataset. However, they were not selected as the final embedded model because they were computationally heavier for the target STM32 system.

SqueezeNet-like lightweight CNN models were also tested as embedded-friendly model candidates. These models showed that a small fire-module-like CNN can give reasonable classification performance with a small model size.

The final embedded model was selected as ST FD-MobileNetV1 from the STM32 AI Model Zoo. This model was selected because it provided a good balance between classification accuracy and STM32 deployment suitability.

2.6. Embedded Deployment Objective

The final objective was to deploy the selected model on the STM32F746G Discovery platform. The board is based on the STM32F746NGH6 microcontroller, which uses an Arm Cortex-M7 core running up to 216 MHz. The microcontroller has limited embedded memory, including about 1 MB Flash memory and about 320 KB internal SRAM. These limitations make model size, activation memory, input image size, and quantization important design factors.

Because of these hardware limitations, a model with high software accuracy is not automatically suitable for the target board. The selected model must be small enough to fit into the available memory and efficient enough to run inference on the microcontroller. For this reason, lightweight model architectures and quantized TensorFlow Lite models were considered during the project.

For deployment, the trained model was converted to TensorFlow Lite format and quantization was applied to reduce model size and memory usage. The final model was then integrated into the STM32 project using STM32 AI tools. In the embedded test setup, preprocessed test images were stored on a microSD card. The STM32 board read each image file, copied the image data to the neural network input buffer, ran inference, and produced a class prediction. The output was observed through UART and, in single-image tests, also through the LCD screen.

The embedded deployment objective can be summarized as follows:

Table 2.3 Embedded deployment and inference steps

Step	Description
Target board	STM32F746G Discovery with Arm Cortex-M7 microcontroller
Hardware limits	About 1 MB Flash and about 320 KB internal SRAM
Model conversion	Convert the trained model to TensorFlow Lite format
Quantization	Reduce model size and memory usage
STM32 integration	Deploy the model using STM32 AI tools
SD card testing	Use preprocessed test images as input
Inference	Run the model on the board
Output	Produce the predicted leaf class

2.7. Summary of Research Objectives

The research objectives of this project are:

● to collect and label tomato plant images from real greenhouse conditions,

● to clean the dataset by removing unclear, low-quality, weakly labelled, and visually ambiguous samples,

● to exclude fruit, stem, and flower categories because the dataset was not suitable for reliable classification of these plant parts,

● to remove back-side leaf images and focus on front-side leaf images for a more consistent classification task,

● to include tuta_leaf in the final class list because of its practical importance,

● to compare different CNN-based model architectures,

● to test preprocessing, image size, interpolation, and augmentation approaches,

● to use EfficientNetB3 as an upper-bound software experiment,

- to evaluate SqueezeNet-like CNNs as custom lightweight model candidates,
- to select ST FD-MobileNetV1 as the final embedded model,
- to convert and quantize the selected model,
- to test the final system on STM32 using SD card input images.

The final goal of the project is to show that a lightweight AI model can be used for front-side tomato leaf health classification on a microcontroller-based system.

3. LITERATURE REVIEW

3.1. Deep Learning in Plant Disease Detection

Plant disease detection is an important research area in precision agriculture. Traditional plant disease diagnosis mostly depends on visual inspection by farmers or experts. This method can be slow, subjective, and difficult to apply to large farms or greenhouses. For this reason, computer vision and deep learning methods have been widely studied for automatic plant disease detection.

Before deep learning became widely used, many image processing methods were based on manually designed features. These methods usually used color, shape, texture, edge information, or threshold-based segmentation. Although these techniques can work in controlled images, they may be sensitive to lighting changes, shadows, background differences, and image quality. This is a limitation for real greenhouse images.

Deep learning models are useful for image-based plant disease classification because they can learn visual features directly from image data. In contrast to handcrafted feature methods, convolutional neural networks can learn useful patterns automatically during training. This makes CNN-based methods suitable for detecting disease symptoms such as spots, color changes, damaged regions, and texture differences on plant leaves.

Several review studies show that deep learning has become a common method for plant disease detection and classification. Saleem et al. reviewed different deep learning architectures used for plant disease detection and explained that CNN-based methods are widely used with metrics such as accuracy, precision, recall, and F1-score. They also reported that the PlantVillage dataset is one of the most widely used datasets in this field [1] .

Many plant disease detection studies use RGB images because RGB cameras are low-cost, easy to use, and suitable for practical agricultural applications. Other imaging methods such as hyperspectral, multispectral, or thermal imaging can provide more detailed information, but they are usually more expensive and more difficult to use in a simple embedded system. For this reason, this project used RGB images collected with mobile phone cameras.

3.2. Tomato Leaf Disease Classification

Tomato leaf disease classification has been studied by many researchers because tomato is an important crop and many disease symptoms are visible on leaves. These symptoms can be detected from RGB images, especially when the disease causes visible color changes, spots, or texture differences.

Zhang et al. studied tomato leaf disease identification using transfer learning. They compared AlexNet, GoogLeNet, and ResNet as backbone models. Their best result was obtained with ResNet, reaching 97.28% accuracy for tomato leaf disease identification. This study shows that deep CNN models can classify tomato leaf diseases with high accuracy. However, models such as ResNet are generally large and may require more computational resources than a microcontroller can provide [2] .

Özbilge et al. proposed a compact CNN model for tomato disease recognition using the PlantVillage tomato dataset. Their model used 10 classes, including nine diseases and one healthy class. The proposed model had only six layers and reached 99.70% accuracy on 9,077 unseen test images. The authors also compared their compact model with larger pre-trained ImageNet models and stated that very deep and complex architectures are not always necessary for tomato disease classification [3].

This result is important for this project because the final system is designed for a microcontroller. A smaller model is more suitable for embedded deployment than a large model, even if the large model gives high accuracy on a computer. Therefore, compact CNN studies support the idea of using lightweight architectures in this project.

3.3. Lightweight CNN Models for Mobile and Embedded Systems

For embedded systems, model size and computational cost are very important. A model that works well on a computer may not be suitable for a microcontroller because microcontrollers have limited Flash memory, RAM, and processing power. For this reason, lightweight CNN architectures are important for embedded AI applications.

Durmuş et al. studied tomato leaf disease detection using AlexNet and SqueezeNet on the PlantVillage tomato dataset. Their goal was to run deep learning models on a mobile computing platform for real-time plant disease detection. They explained that the detection code must be lightweight in terms of computation cost and RAM usage when it is used on a robot platform [4].

In their results, AlexNet reached higher accuracy, but it had a much larger model size. SqueezeNet had slightly lower accuracy, but it was much smaller and faster. This result shows the trade-off between accuracy and model size. For embedded systems, a slightly lower accuracy can be acceptable if the model becomes much smaller and easier to deploy.

MobileNetV2 is another lightweight architecture designed for mobile and resource-limited platforms. Zaki et al. used MobileNetV2 for tomato leaf disease classification using images from the PlantVillage dataset. Their study used four classes, and the input images were resized to 224×224 pixels. The model achieved more than 90% accuracy [5].

MobileNet-based models are important for embedded applications because they use efficient convolution operations to reduce computation. In this project, ST FD-MobileNetV1 from the STM32 AI Model Zoo was selected as the final embedded model because it provided a good balance between classification accuracy and STM32 deployment suitability.

3.4. High-Capacity CNN Models as Reference Experiments

Some deep learning models are designed mainly for high accuracy rather than small memory usage. EfficientNet is one example of these high-capacity architectures. Such models can be useful for understanding the possible performance limit of a dataset, but they may not be suitable for small microcontrollers.

In this project, EfficientNetB3 was tested as a reference model. This experiment was useful because it showed that the cleaned front-side leaf dataset could reach higher software-level accuracy when a larger model was used. However, EfficientNetB3 was not selected as the final embedded model because it is computationally heavier than the target STM32 system can comfortably support.

Therefore, EfficientNetB3 was used as an upper-bound experiment. The final model selection was based not only on accuracy, but also on model size, quantization, memory usage, and STM32 deployment feasibility.

3.5. Object Detection and Disease Localization Studies

Some recent studies focus not only on classifying the leaf image, but also on detecting and localizing disease regions. Object detection models such as YOLO can locate diseased areas on the leaf and predict the disease class at the same time. These methods are useful when the aim is to show the exact position of the infected region.

Umar et al. proposed a tomato disease recognition system using CNN and an improved YOLOv7 model. Their study focused on detecting and categorizing tomato leaves in field conditions. The paper also discusses the difficulty of real-world tomato leaf disease recognition, such as similar disease symptoms and the lack of enough annotated real-world images [6].

Sun et al. proposed E-TomatoDet, a tomato leaf disease detection model that combines global and local feature extraction. Their model uses CSWinTransformer, a Local

Feature Enhance Pyramid, and a multi-kernel module. The model reached high detection performance on a tomato leaf disease dataset. This shows that modern detection models can achieve strong performance, especially when they combine local and global features [7].

Sowmya and Guruprasad proposed a system using Inception-v4 CNN and YOLOv8 for tomato plant disease detection. In their approach, Inception-v4 was used for classification and YOLOv8 was used for real-time localization. The study reported strong classification and detection performance [8].

These studies show that object detection models can be powerful for disease localization. However, they are generally more complex than simple image classification models. Since this project aims to run the final system on an STM32 microcontroller, the final implementation focuses on lightweight image classification instead of full object detection.

3.6. Research Gap and Relation to This Project

The reviewed studies show that deep learning methods can classify tomato leaf diseases with high accuracy. Many studies use public datasets such as PlantVillage, and many of them report high accuracy values using CNN, transfer learning, MobileNet, SqueezeNet, EfficientNet, or YOLO-based models.

However, there are still some limitations for microcontroller-based applications. First, many high-accuracy models are too large for small embedded systems. Second, many studies are tested on clean public datasets, where the images usually have simple backgrounds and clear symptoms. Third, many studies focus on software-level performance and do not include microcontroller deployment.

This project addresses these points by focusing on a complete embedded AI workflow using real greenhouse image data. A custom tomato plant dataset was collected from greenhouse conditions and labelled manually using CVAT. The dataset was cleaned several times to remove unclear and unreliable samples. Several model architectures were tested, including EfficientNetB3, SqueezeNet-like lightweight CNNs, and ST FD-MobileNetV1.

The final system was designed for five front-side leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf. Fruit, stem, and flower categories were not included in the final classifier because the collected dataset was not suitable for reliable classification of these plant parts. Back-side leaf images were also removed because their sample number was lower and their visual appearance was different from front-side leaves.

The main contribution of this project is not only training a tomato leaf classifier, but also building a practical pipeline from real greenhouse image collection and CVAT labelling to model training, quantization, and STM32-based inference.

The comparison in Table 3.1 shows that previous studies achieved strong results using CNN-based models for tomato leaf disease classification. However, many of these studies used clean public datasets and focused mainly on software-level performance. In contrast, this project used a custom dataset collected from real greenhouse conditions. The final model was also deployed and tested on an STM32 microcontroller using SD card input images. Therefore, the main difference of this project is the combination of real greenhouse data preparation, lightweight model selection, quantization, and embedded STM32 inference.

Table 3.1 Comparison of related studies and this project

Study / System	Model Type	Dataset Type	Main Focus	Embedded Test
Zhang et al. [2]	AlexNet, GoogLeNet, ResNet	Public tomato leaf dataset	High software-level tomato disease classification accuracy	No
Özbilge et al. [3]	Compact CNN	PlantVillage tomato dataset	Compact CNN for tomato disease recognition	No
Durmuş et al. [4]	AlexNet and SqueezeNet	PlantVillage tomato dataset	Lightweight model selection for mobile/robot use	Mobile computing platform
Zaki et al. [5]	MobileNetV2	PlantVillage tomato dataset	Mobile-oriented tomato leaf disease classification	No
This project	ST FD-MobileNet V1	Custom real greenhouse tomato leaf dataset	Front-side tomato leaf classification and STM32 deployment	Yes

4. DESIGN

The proposed system is designed as a microcontroller-based artificial intelligence system for tomato leaf health classification under real greenhouse conditions. The design includes both machine learning and embedded system parts. The main purpose is to create a complete workflow from real greenhouse image collection to STM32-based inference.

The general design workflow is shown in Figure 4.1.

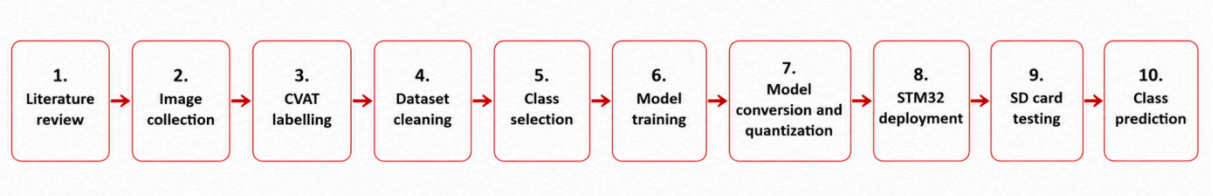


Figure 4.1 The general design workflow

The final system does not use a live camera directly on the STM32 board. Instead, preprocessed test images are stored on a microSD card and used as input for inference. This design decision reduces the complexity of the prototype and makes the embedded tests more repeatable. Camera integration and real-time image capture can be added as future work.

The design was shaped by two main requirements. First, the model had to classify real greenhouse tomato leaf images with acceptable accuracy. Second, the model had to be lightweight enough for STM32 deployment. Therefore, model size, memory usage, input image size, quantization, and STM32 deployment feasibility were considered together during the design process.

4.1. Realistic Constraints and Conditions

The design of the project was affected by several realistic constraints. These constraints include economic, environmental, ethical, health, safety, manufacturability, sustainability, social, and technical limitations.

4.1.1. Economic Constraints

One of the main economic goals of the project is to use a low-cost embedded platform instead of a computer or cloud server during inference. A microcontroller-based system can be cheaper and more practical for greenhouse applications if the model can run locally on the board.

The project uses an STM32F746G Discovery board, a microSD card, and free or student-accessible software tools such as Google Colab, TensorFlow/Keras, STM32CubeMX, STM32CubeIDE, X-CUBE-AI, and STM32 AI Model Zoo. Since the model inference is performed locally on the microcontroller, the system does not require a continuous internet connection or a cloud-based AI service during inference.

This makes the design more suitable for small-scale greenhouse monitoring applications.

4.1.2. Environmental Constraints

Early detection of plant health problems can help reduce unnecessary pesticide use. If a disease or pest problem is detected earlier, the farmer can apply treatment only when it is needed. This can reduce chemical waste and environmental damage.

The proposed system also uses a low-power embedded platform. Compared to running inference on a computer or server, a microcontroller-based system can consume less energy during operation. This is useful for future greenhouse or field applications where energy efficiency is important.

4.1.3. Sustainability

The project supports sustainable agriculture by helping plant health monitoring become more automatic and accessible. A local embedded AI system can reduce the need for continuous manual inspection and can support early decision-making.

The system is also designed as a modular pipeline. The dataset, model, and embedded implementation can be improved separately. For example, more images can be added to the dataset, the model can be retrained, or the embedded system can be upgraded with a camera module in the future.

4.1.4. Manufacturability

The current project is implemented as a prototype using a development board. This is suitable for testing the feasibility of the design. The STM32F746G Discovery board already includes useful hardware components such as the microcontroller, memory interfaces, LCD screen, USB connection, and microSD card interface.

For a real product, the design could be transferred to a custom PCB with only the required components. The SD card interface, power input, display, or communication interface would be selected according to the final application. Since the project uses STM32 tools and C-based generated code, the software side is also suitable for further embedded product development.

4.1.5. Ethical Considerations

The dataset used in this project includes tomato plant images. No human images or personal data are used. Therefore, there is no direct privacy issue in the dataset.

However, the system output should be considered as a decision support result, not as a final agricultural diagnosis. A wrong prediction may cause wrong treatment decisions. For this reason, the system should be improved and tested with more real greenhouse data before being used in a real agricultural decision process.

4.1.6. Health and Safety

The project uses low-voltage electronic hardware. Basic laboratory safety rules are enough during development. The STM32 board is powered through USB, and no high-voltage circuit is used.

From the agricultural side, the health-related issue is indirect. If the model gives a wrong prediction, it may lead to unnecessary or incorrect pesticide use. Therefore, the model should not be used alone for critical decisions without expert confirmation, especially in a real farm environment.

4.1.7. Social and Practical Constraints

In many farms and greenhouses, expert inspection may not always be available. A low-cost AI-based monitoring system can help farmers by giving early warnings about plant health problems.

However, the system must be simple to use. If the system requires too much technical knowledge, it may not be practical for farmers. For this reason, the final long-term design should include a simple user interface, clear prediction results, and easy dataset/model update options.

4.1.8. Technical Constraints

The most important technical constraint is the limited memory and processing power of the microcontroller. Large CNN models can give high accuracy on a computer, but they may not fit into the Flash and RAM limits of an embedded board.

The STM32F746G Discovery board uses the STM32F746NGH6 microcontroller with an Arm Cortex-M7 core. The board is powerful for many embedded applications, but it still has limited internal memory compared with a computer or cloud platform. The microcontroller has about 1 MB Flash memory and about 320 KB internal SRAM. Therefore, the selected neural network had to fit within embedded memory limitations together with the input buffer, output buffer, and intermediate activation memory.

The input image size was another technical constraint. Larger input images preserve more disease and pest details, but they increase memory usage and computation cost. Smaller input images reduce the cost, but small symptoms such as flies or trips may become difficult to detect after resizing.

This trade-off affected the final model design. Different input sizes and model architectures were tested, and the final model was selected by considering both accuracy and embedded compatibility.

4.2. Cost of the Design

The prototype was designed using mostly available hardware and free software tools. Therefore, the cost of the project is mainly related to the development board and storage device.

Table 4.1 Cost of the Design

Component / Tool	**Quantity**	**Purpose**	**Cost Type**
STM32F746G	1	Embedded inference platform	Hardware cost
microSD Card	1	Storage for test images	Low-cost hardware
USB Cable	1	Power and programming	Low-cost hardware
Personal Computer	1	Dataset preparation, training, coding, and STM32 development	Existing equipment
Mobile Phone Camera	1	Dataset image collection	Existing equipment
CVAT	-	Data labelling	Free Software
Google Colab	-	Model training	Free Software
TensorFlow / Keras	-	Model development	Free software
STM32CubeMX / STM32CubeIDE	-	Embedded software development	Free software
X-CUBE-AI	-	AI model conversion for STM32	Free software
STM32 AI Model Zoo	-	Embedded-oriented model selection	Free software resource

The cost table shows that the system was developed mostly with accessible tools and available hardware. This is important because the goal of the project is not only to create a working AI model, but also to show that embedded AI-based plant health monitoring can be developed with a low-cost prototype structure.

4.3. Engineering Standards

Several engineering standards were considered during the design and implementation of this project. Since the project includes dataset preparation, software development, artificial intelligence model training, model evaluation, quantization, and embedded deployment, the relevant standards are mainly related to software life-cycle processes, system life-cycle processes, software quality, testing, artificial intelligence systems, verification and validation, and ethical system design.

ISO/IEC/IEEE 12207:2026 was considered as a reference for software life-cycle processes [9]. In this project, the software workflow included dataset preparation scripts, model training notebooks, TensorFlow Lite conversion, quantization, STM32 integration, and test code. These steps are related to software development, operation, maintenance, and improvement processes. Therefore, this standard was relevant for understanding the software process structure of the project.

ISO/IEC/IEEE 15288:2023 was also relevant because the project was not only a software project [10]. It included a microcontroller board, SD card input, embedded inference code, a trained AI model, and test procedures. Therefore, the project can be considered as a small system that combines hardware and software components. The life-cycle approach of this standard is related to the design, implementation, integration, verification, and testing stages of the project.

ISO/IEC 25010:2023 was used as a quality reference for evaluating the software and system quality aspects of the project [11]. The most important quality characteristics for this project were functional suitability, performance efficiency, reliability, and maintainability. Functional suitability was related to correct tomato leaf classification. Performance efficiency was

related to model size, RAM usage, Flash usage, and inference feasibility on STM32. Reliability was considered through software-level testing and STM32 SD card testing.

ISO/IEC/IEEE 29119-1:2022 was considered for software testing concepts [12]. In this project, the dataset was divided into training, validation, and test subsets. The final model was evaluated in the software environment, and then the deployed model was tested on the STM32 board using SD card input files. These testing stages are consistent with the idea of using defined test processes and test evidence.

For the artificial intelligence part of the project, ISO/IEC 22989:2022 and ISO/IEC 23053:2022 were considered [13], [14]. ISO/IEC 22989:2022 provides artificial intelligence concepts and terminology. ISO/IEC 23053:2022 provides a framework for describing AI systems that use machine learning. These standards are related to the dataset, model training, model evaluation, and inference workflow used in this project.

ISO/IEC 5338:2023 was also relevant because it describes AI system life-cycle processes [15]. The project followed a simplified AI life-cycle: data collection, labelling, dataset cleaning, model training, evaluation, quantization, deployment, and testing. Therefore, this standard was relevant to the overall AI development flow of the project.

IEEE 1012-2024 was considered as a reference for verification and validation [16]. The model was first verified in the software environment using the test dataset. Then, the deployed model was validated on the STM32 board using the SD card test. The STM32 test was important because it showed whether the final system worked in the target embedded environment.

Finally, ISO/IEC/IEEE 24748-7000:2022 was considered from an ethical system design perspective [17]. The proposed system is intended to support plant health monitoring, not to replace expert agricultural decisions completely. Since wrong predictions may cause incorrect treatment decisions, the system should be used as a decision-support tool. This is especially important for agricultural systems where model outputs may influence pesticide or treatment decisions.

Table 4.2 summarizes the engineering standards related to this project.

Table 4.2 Engineering standards related to the project

Standard	Area	Relation to this project
ISO/IEC/IEEE 12207:2026	Software life cycle	Dataset scripts, model training, conversion, quantization, and test code
ISO/IEC/IEEE 15288:2023	System life cycle	Hardware and software integration with STM32
ISO/IEC 25010:2023	System and software quality	Functional suitability, performance efficiency, reliability, and maintainability
ISO/IEC/IEEE 29119-1:2022	Software testing	Train/validation/test split, software test, and STM32 test
ISO/IEC 22989:2022	AI terminology	AI and machine learning concepts
ISO/IEC 23053:2022	ML-based AI system framework	Dataset, model training, evaluation, and inference workflow
ISO/IEC 5338:2023	AI system life cycle	Data collection, labelling, cleaning, training, deployment, and testing
IEEE 1012-2024	Verification and validation	Software verification and STM32 validation
ISO/IEC/IEEE 24748-7000:2022	Ethical system design	Decision-support role and responsible use

4.4. Details of the Design

4.4.1. Dataset Design

The dataset was collected from tomato plants in real greenhouse conditions. Images were taken using mobile phone cameras. After collection, the images were labelled manually using CVAT.

The original dataset included different plant parts such as leaves, fruits, stems, and flowers. However, these categories were not balanced. Healthy fruit samples were available in large numbers, but unhealthy fruit samples were very limited. Also, no unhealthy stem or flower samples were available in the collected dataset. Including these non-leaf categories could produce misleading accuracy values because the model could learn the dominant healthy classes and increase the overall accuracy artificially.

For this reason, the final model was focused on leaf-based classes. In addition, back-side leaf images were removed from the final dataset. Back-side images had fewer samples and their visual appearance was different from front-side leaf images. This difference increased the variation inside the dataset and made the classification task more confusing.

In practical greenhouse inspection, leaves are usually checked from the upper/front side first. Disease and pest symptoms are also generally easier to observe on the upper surface of the leaf. Therefore, the final classification system was designed with front-side leaf images only.

The final classes are given in Table 4.3.

Table 4.3 Final selected front-side leaf classes

Class No	Class Name
1	flies_leaf
2	healthy_leaf
3	nutdef_leaf
4	trips_leaf
5	tuta_leaf

The tuta_leaf class had fewer samples than the other classes. However, it was included in the final dataset because it is an important pest-related class in tomato farming. Removing this class would make the classification task easier, but it would reduce the practical value of the system.

Some of the final leaf classes have similar visual appearances. This similarity was one of the reasons why dataset cleaning and per-class evaluation were important. Example images from visually similar classes are shown in Figure 4.2.



Figure 4.2 Examples of visually similar leaf classes: trips_leaf and flies_leaf

4.4.2. Model Design

The model design process was developed iteratively during the project. The aim was not only to train one model and report its accuracy. Instead, different model families, class combinations, input sizes, preprocessing methods, and augmentation settings were tested step by step. After each experiment, the result was examined and the next experiment was planned according to the observed limitations.

In the early stage, larger CNN architectures such as ResNet-based models were tested as baseline experiments. These models were useful for understanding whether the dataset contained learnable visual patterns. However, they were not suitable as final embedded models because they were relatively heavy for microcontroller deployment.

After the early baseline experiments, simpler CNN structures were also tested. One of the motivations for this step was the compact CNN idea in the literature, where smaller CNN models can sometimes perform well for tomato disease classification without using very deep architectures [3]. However, the early compact CNN trials did not provide sufficient performance on the collected greenhouse dataset. This showed that the real dataset was more difficult than clean public datasets and that model design alone was not enough without improving dataset quality.

During the project, different class structures were also tested. Earlier experiments included more classes, including different plant parts and back-side leaf images. However, these settings caused class imbalance and additional visual variation. Some classes had very few samples, and some categories were not suitable for reliable classification. For this reason, the dataset scope was gradually narrowed. The final decision was to use five front-side leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf.

Input image size was another important part of the model design. Smaller input sizes such as 128×128 and 160×160 reduced memory and computation cost, but they also caused loss of small disease and pest details. This was especially important for pest-related classes, where the visible symptom may be a small insect or a small mark on the leaf. Larger input sizes preserved more visual details but increased memory usage. For this reason, several input sizes such as 160×160, 192×192, 224×224, and 300×300 were tested during the model development process.

Different preprocessing and resizing methods were also evaluated. Standard resizing, bilinear interpolation, area interpolation, and sharpening-based preprocessing were tested to understand whether small symptoms could be preserved better after resizing. In addition, augmentation settings such as flipping, rotation, brightness changes, contrast changes, and stronger augmentation trials were tested. These experiments showed that preprocessing and

augmentation can affect training, but they cannot fully solve problems caused by unclear labels or weak visual symptoms.

In the later stage, EfficientNetB3 was tested as a high-capacity software reference model. This model was not intended for STM32 deployment. Its purpose was to estimate the upper performance limit of the cleaned dataset when a stronger model was used. These experiments helped to show that the final front-side leaf dataset had learnable visual patterns.

SqueezeNet-like lightweight CNN models were then tested as custom embedded candidates. These models used fire-module-like blocks and were designed to reduce the number of parameters while keeping useful feature extraction capability. The SqueezeNet-like experiments were important because they showed that a small custom model could give reasonable performance for the tomato leaf classification task. However, the final ST FD-MobileNetV1 model gave a better balance between accuracy and STM32 deployment suitability.

The final model was selected as ST FD-MobileNetV1 from the STM32 AI Model Zoo. This model was preferred because it was designed for embedded AI applications and had a more direct deployment path for STM32. The model input size was selected as 224×224 pixels, and the output layer was adapted for five front-side tomato leaf classes.

Table 4.4 summarizes the main model design stages.

Table 4.4 Main stages of the model design process

Stage	Model / Design Trial	Purpose	Design Outcome
Early baseline	ResNet-based models	Understand dataset learnability	Useful baseline, but heavy for STM32
Compact CNN trial	Simple / compact CNN structure	Test small CNN idea from literature	Performance was not sufficient on real greenhouse data
Class structure trials	13-class, 7-class, 6-class, and 5-class versions	Find a reliable class structure	Final scope became five front-side leaf classes
Input size trials	128×128, 160×160, 192×192, 224×224, 300×300	Balance detail preservation and memory cost	224×224 selected for final model
Preprocessing trials	Bilinear resize, area interpolation, sharpening	Preserve small symptoms after resizing	No consistent improvement alone
Augmentation trials	Flip, rotation, brightness, contrast, stronger augmentation	Improve generalization	Useful, but dataset quality remained critical
EfficientNetB3	High-capacity CNN	Estimate upper software-level performance	Used as upper-bound reference
SqueezeNet-like CNN	Fire-module-like lightweight CNN	Test custom embedded candidate	Reasonable lightweight result
ST FD-MobileNetV1	STM32 AI Model Zoo model	Final embedded model candidate	Selected as final model

The final model design was therefore based on both classification performance and embedded feasibility. The selected model had to classify real greenhouse leaf images with acceptable accuracy, but it also had to be small enough to be converted, quantized, and tested on the STM32F746G platform.

4.4.3. Input Image Design

The input image size was an important design parameter. Larger images preserve more details about disease spots and pest symptoms. However, larger images also require more memory and computation.

During the project, different input sizes were tested, including 128×128, 160×160, 192×192, 224×224, and larger input sizes for high-capacity models. Very small input sizes reduced computation cost but caused loss of visual details. This was especially important for classes where the difference was based on small insects or small disease marks.

Several resizing and preprocessing approaches were also tested. These included standard bilinear resizing, area interpolation, and sharpening-based preprocessing. These experiments were useful to understand whether symptom visibility could be improved after resizing. However, they did not provide a clear and consistent improvement over the final model selection.

For the final embedded model, 224×224 input resolution was used because it provided a better balance between visible detail and model compatibility.

4.4.4. Model Conversion and Quantization Design

After training, the selected model was converted to TensorFlow Lite format. Quantization was applied to reduce the model size and make the model more suitable for microcontroller deployment.

The quantized model was then used for embedded integration. This step is important because a normal trained model may be too large or inefficient for a microcontroller. Quantization reduces memory usage and helps the model run on embedded hardware.

Before STM32 deployment, the quantized model was also tested in the software environment. This step was used to check whether quantization caused a significant accuracy drop.

4.4.5. Embedded System Design

The embedded system is based on the STM32F746G Discovery board. The trained and quantized model is integrated into the STM32 project using STM32 AI tools. The board reads preprocessed input images from a microSD card and runs inference using the deployed model.

The simplified embedded workflow is shown in Figure 4.3.

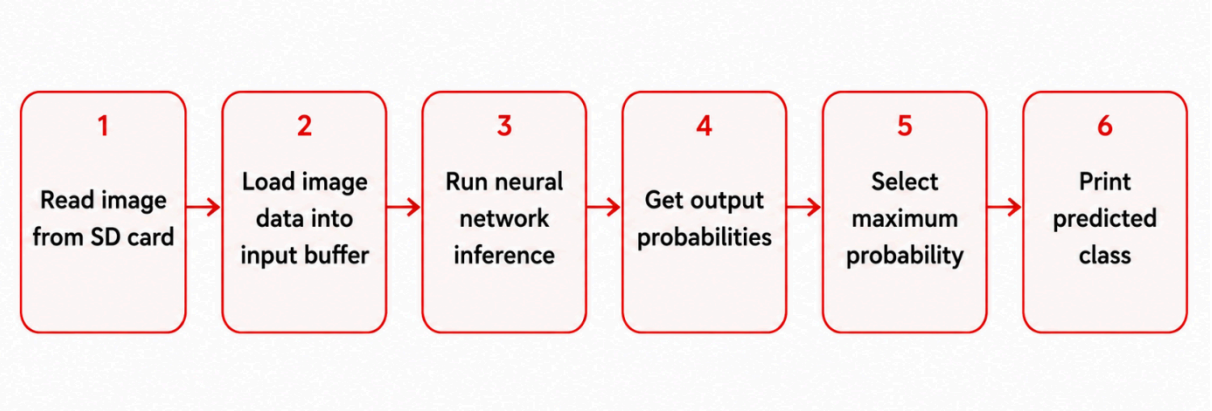


Figure 4.3 STM32 Inference Workflow

The output can be shown through UART or on the LCD screen. In earlier tests, a sample image was converted into a C header file and embedded directly into the project. Later, the design moved toward SD card-based testing because it allows many test images to be evaluated more easily without rebuilding the project.

4.4.6. Testing Design

The model was tested in three main environments. First, it was tested in Google Colab using the test dataset. This gave the software-level test accuracy and confusion matrix. Second, the quantized model was tested before deployment. Third, the model was tested on the STM32 board using SD card input images.

The SD card test setup allowed multiple images to be tested on the board. This was important because a single image test is not enough to evaluate the real performance of the embedded system.

Table 4.5 Final testing design

Test Type	Purpose
Keras model test	Evaluate original trained model
TFLite model test	Check quantized model accuracy
STM32 SD card test	Verify embedded inference pipeline
Per-class results	Understand which classes are difficult
Confusion matrix	Analyze class-to-class mistakes

This design makes it possible to evaluate both model accuracy and embedded deployment success.

5. METHODS

5.1. Overall Methodology

The methodology of this project consists of literature review, greenhouse image collection, manual image labelling, dataset cleaning, class selection, image preprocessing, model development, model conversion, quantization, and STM32 deployment. The aim was to build a complete workflow from real greenhouse image collection to microcontroller-based tomato leaf health classification.

Before the implementation stage, related studies on plant disease detection, tomato leaf classification, lightweight CNN models, MobileNet-based architectures, and embedded AI systems were reviewed. This helped to define the project scope and to understand the importance of dataset quality, model size, memory usage, quantization, and deployment feasibility.

The general workflow of the project is shown in Figure 5.1.

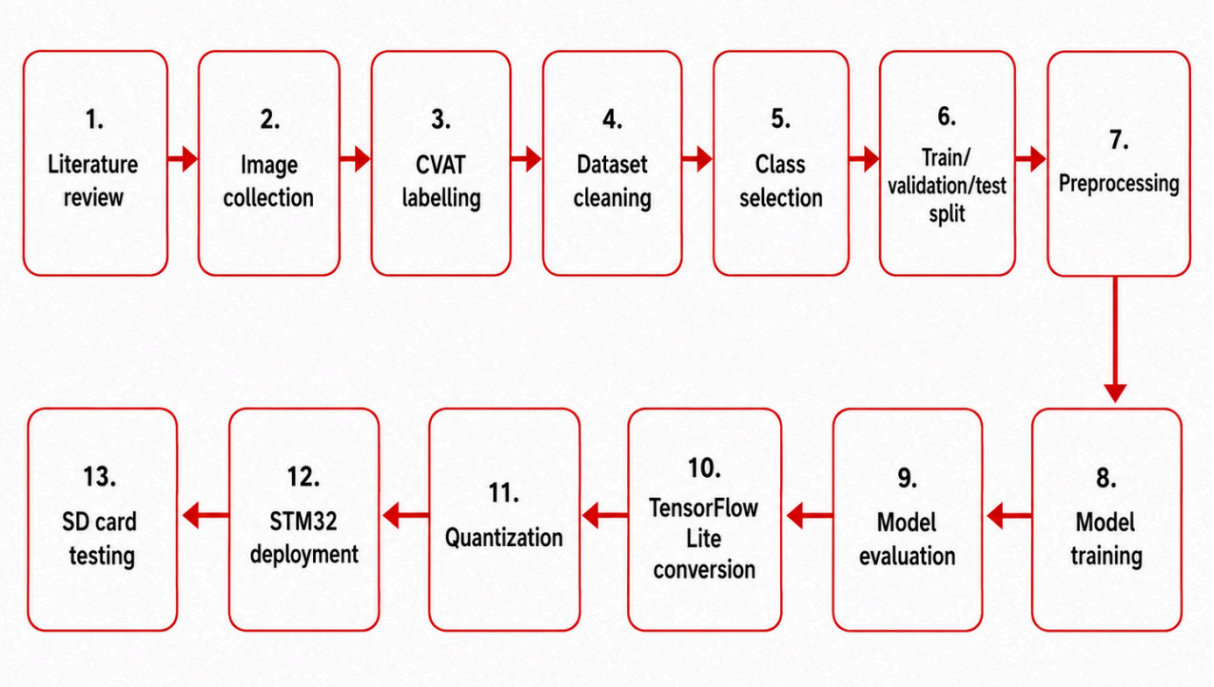


Figure 5.1 Overall methodology workflow

The project was developed step by step. First, different class structures and model families were tested to understand the dataset and the classification problem. Later, the dataset was cleaned and simplified to obtain a more consistent classification task. The final implementation focused on five front-side tomato leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf.

Several model groups were tested during the project. Early CNN and ResNet-based models were used to understand the difficulty of the dataset. EfficientNetB3 was used as a high-capacity software reference model. SqueezeNet-like lightweight CNN models were tested as custom embedded candidates. Knowledge distillation was also tested using a stronger teacher model and a smaller student model. Finally, ST FD-MobileNetV1 from the STM32 AI Model Zoo was selected as the final embedded model because it provided a good balance between accuracy and STM32 deployment suitability.

5.2. Dataset Collection

The dataset used in this project was collected manually from tomato plants in real greenhouse conditions. Images were taken with mobile phone cameras during a period of approximately ten weeks. Different project members collected images from the greenhouse in different weeks.

The dataset was not taken from a ready and clean public dataset. This was one of the main differences of the project. Real greenhouse images included different lighting conditions, different camera angles, different camera distances, different leaf positions, background objects, shadows, and different image qualities. Therefore, the dataset was more realistic, but also more difficult than controlled public datasets.

At the beginning of the project, the collected images included different tomato plant parts such as leaves, fruits, stems, and flowers. However, the final model focused only on front-side leaf images. This decision was made because the collected dataset was not suitable for reliable classification of fruit, stem, and flower health conditions. Unhealthy fruit

samples were very limited, and no unhealthy stem or flower samples were available in the collected dataset.

Another difficulty was the limited availability of diseased or pest-affected samples. Since the tomato plants were grown in controlled greenhouse conditions, most plants were healthy most of the time. Some disease or pest symptoms appeared only in certain weeks. In some weeks, several plants showed similar symptoms, but after treatment and greenhouse maintenance, these symptoms decreased in the following days or weeks. Therefore, the dataset naturally changed from week to week, and obtaining balanced class data was difficult.

Another important point is that the dataset was not fixed during the whole project. While model experiments were being performed, new greenhouse images were still being collected, labelled, checked, and cleaned. Therefore, the dataset gradually grew and changed during the development process. Some experiments were performed on earlier dataset versions, while later experiments used cleaner and more updated versions of the dataset. This is one of the reasons why the model results changed over time. The final experiments were performed using the cleaned front-side leaf dataset after the main class selection and dataset cleaning decisions were completed.

Some images were also difficult to capture because of physical and environmental limitations. Upper leaves were sometimes hard to reach, and the angle of sunlight made it difficult to take clear images from certain positions. These problems caused variations in brightness, focus, distance, and image quality.

Figure 5.2 shows a general view of the greenhouse used for dataset collection.



Figure 5.2 General view of the greenhouse used for dataset collection

5.3. Image Labelling and Dataset Organization

After image collection, the images were labelled manually using CVAT. This step was necessary because the dataset was collected specifically for this project and the raw images did not have class labels.

The original labels included different plant parts, different leaf positions, and different viewing sides. For example, some early labels included upper leaf, middle leaf, lower leaf, and back-side leaf categories. During the project, these detailed labels were merged into more general class names when they represented the same health condition.

The labelling process was not completed in a single step. Images collected by different project members were labelled and then reviewed again. Some labelling decisions were discussed with the project supervisor and academic partners. After these discussions,

labelled data was shared among the students for checking, cleaning, and correction before being added back to the main dataset pool.

The final dataset was organized in a directory-based format. Each class had its own folder, and the images inside each folder belonged to that class. This format made it possible to load the data using TensorFlow/Keras data loading functions. It also made the final STM32 SD card test organization easier.

The class order was kept the same during training, evaluation, TensorFlow Lite conversion, SD card export, and STM32 inference. This was necessary because a class order mismatch could produce wrong class names even if the model output index was correct.

Figure 5.3 shows an example of manual image labelling in CVAT.

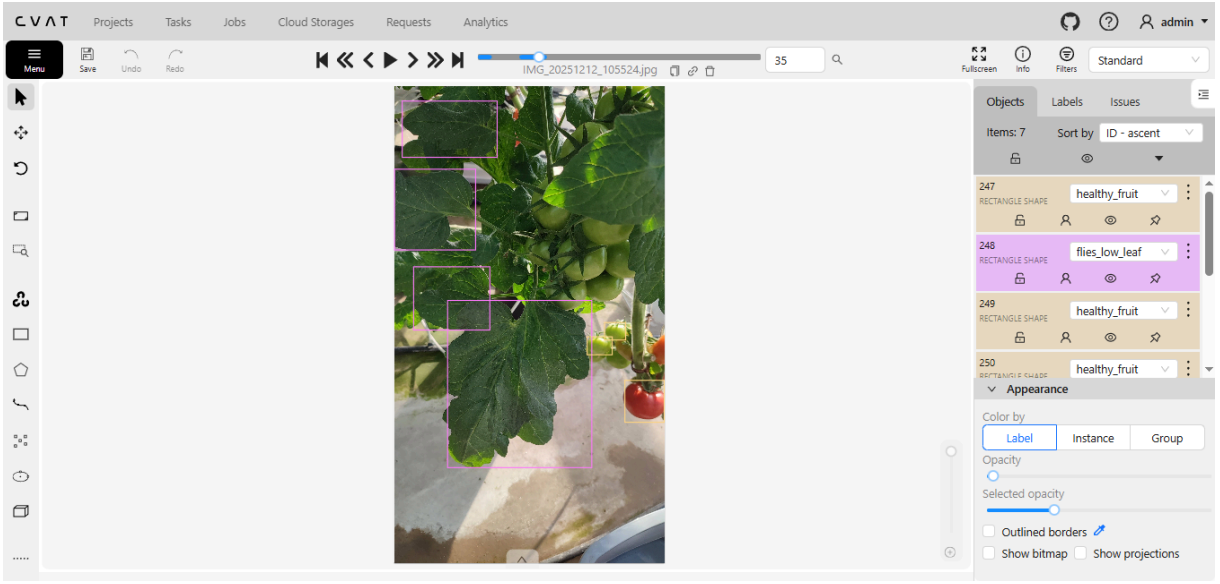


Figure 5.3 Example of manual image labelling in CVAT

Table 5.1 shows the original leaf labels and their final class names.

Table 5.1 Original labels to final classes mapping

Original Labels	Final Class
flies_up_leaf, flies_mid_leaf, flies_low_leaf	flies_leaf
flies_up_back_leaf, flies_mid_back_leaf, flies_low_back_leaf	flies_back_leaf
healthy_up_leaf, healthy_mid_leaf, healthy_low_leaf	healthy_leaf
healthy_up_back_leaf, healthy_mid_back_leaf, healthy_low_back_leaf	healthy_back_leaf
nutdef_up_leaf, nutdef_mid_leaf, nutdef_low_leaf	nutdef_leaf
nutdef_up_back_leaf, nutdef_mid_back_leaf, nutdef_low_back_leaf	nutdef_back_leaf
trips_up_leaf, trips_mid_leaf, trips_low_leaf	trips_leaf
trips_up_back_leaf, trips_mid_back_leaf, trips_low_back_leaf	trips_back_leaf
tuta_up_leaf, tuta_mid_leaf, tuta_low_leaf, tuta_up_back_leaf, tuta_mid_back_leaf, tuta_low_back_leaf,	tuta_leaf

5.4. Dataset Cleaning

Dataset cleaning was one of the most important parts of the methodology. In the first experiments, the dataset contained unclear, low-quality, and visually ambiguous images. Some images were low-resolution, some leaves were out of focus, and some pest symptoms were too weak to see clearly.

These problems affected model performance. For this reason, the dataset was reviewed several times. Low-quality images, unclear samples, and unreliable labels were removed. For

example, some leaves had only one very small visible insect. These samples were removed because the class information could become unclear after resizing to the model input size.

Another important check was done by resizing sample images to common model input sizes such as 224×224 and 192×192 pixels. In some resized images, even a human observer could not clearly see the disease or pest symptoms. These samples were considered unsuitable for training because the model would also have difficulty learning from them.

The cleaning process continued during model training experiments. When a model showed repeated confusion or when some images looked suspicious during manual review, the dataset was checked again. In many training notebooks, incorrectly predicted test images were saved into a separate output folder or file after evaluation. These misclassified samples were then manually reviewed. During this review, some images were found to be truly difficult or visually ambiguous. In some cases, the image was too unclear after resizing, and in other cases, the original label was not reliable enough. If an image was wrongly labelled, its class was corrected. If the visual evidence was too weak or ambiguous, the image was removed from the dataset. This process helped improve dataset quality and made later experiments more reliable. Some images were removed, some images were added, and some class labels were corrected. Since the dataset continued to grow during the project, dataset cleaning was repeated several times instead of being applied only once. Therefore, dataset preparation was an iterative process rather than a single preprocessing step.

The aim of this step was not only to have a large dataset, but to have a more reliable dataset. After cleaning, the model results became more stable and meaningful.

Figure 5.4 shows the effect of resizing on a pest-related flies_leaf sample.



Figure 5.4 Effect of resizing on a flies_leaf sample

Figure 5.5 shows another resized leaf example.



Figure 5.5 Effect of resizing on a trips_leaf sample

5.5. Class Selection and Final Dataset Split

The final model was trained using five front-side leaf classes. Fruit, stem, and flower categories were excluded from the final model because the collected dataset was not suitable for reliable classification of these plant parts. Unhealthy fruit samples were very limited, and no unhealthy stem or flower samples were available.

Back-side leaf images were also removed from the final dataset. Their sample number was lower than front-side leaf images, and their visual appearance was different. Back-side

images had different color, texture, and vein appearance. This increased the variation inside the dataset and made the classification task more confusing.

In practical greenhouse inspection, leaves are usually checked from the upper/front side first. Disease and pest symptoms are also generally easier to observe on this surface. Therefore, the final dataset was restricted to front-side leaf images only.

The final classes were:

- flies_leaf,
- healthy_leaf,
- nutdef_leaf,
- trips_leaf,
- tuta_leaf.

Example images from the final five front-side leaf classes are shown in Figure 5.5. These examples illustrate the visual differences between the selected classes and also show why symptom visibility is important for the classification task.

Figure 5.6 Example images from the final five front-side leaf classes

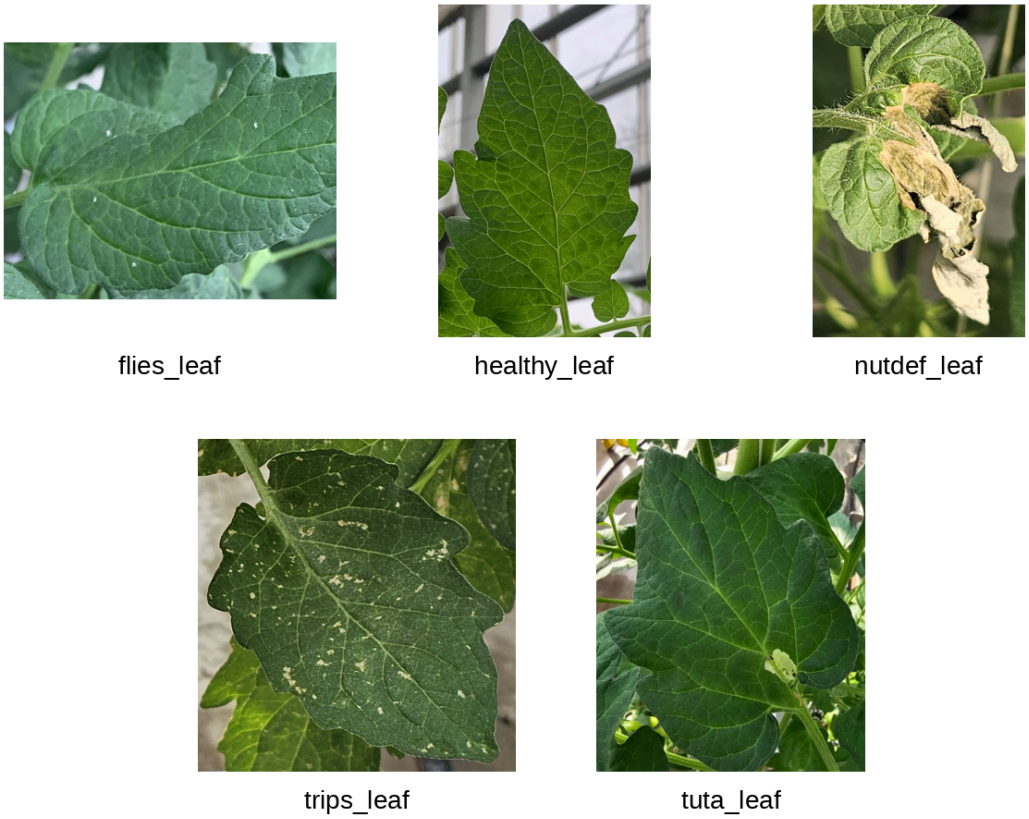


Figure 5.6 Example images from the final five front-side leaf classes

The tuta_leaf class had fewer samples than the other classes. However, it was kept in the final dataset because it is an important pest-related problem in tomato farming. Removing this class would make the classification task easier, but it would reduce the practical value of the system.

After class selection and dataset cleaning, the dataset was divided into training, validation, and test sets. The training set was used to update the model weights. The validation set was used to monitor model performance during training. The test set was used only for final evaluation.

The final split was 70% training, 15% validation, and 15% testing. The split operation was done separately for each class so that all classes were represented in each subset.

Table 5.2 shows the final dataset distribution.

Table 5.2 The final dataset distribution

Class	Train	Validation	Test	Total
flies_leaf	1397	300	300	1997
healthy_leaf	3027	649	649	4325
nutdef_leaf	660	141	142	943
trips_leaf	1393	299	299	1991
tuta_leaf	68	15	15	98
Total	6545	1404	1405	9354

The table shows that the dataset was not perfectly balanced. The healthy_leaf class had the highest number of samples, while tuta_leaf had the lowest number of samples. For this reason, tuta_leaf results should be interpreted carefully.

5.6. Image Preprocessing

All images were resized to a fixed input size before being given to the model. Different input sizes were tested during the project, including 128×128, 160×160, 192×192, 224×224, and 300×300.

Smaller input sizes reduced computation cost, but they also removed small visual details. This was a problem especially for pest-related classes where the difference depended on small insects or small leaf marks. Larger input sizes preserved more details, but they required more memory and computation.

The number of trainable parameters does not necessarily increase when only the input image size is changed. In convolutional neural networks, the parameter count is mainly determined by the number of filters and layers. However, larger input images increase the input buffer size, activation memory, computation cost, and inference time. Therefore, the

input image size was selected by considering both symptom visibility and embedded memory limitations.

Table 5.3 Effect of input image size on memory and visual detail

Input Size	Input Pixels	Raw RGB Input Size	Expected Effect	Observation
128×128	16,384	49,152 bytes	Lowest memory and computation cost	Small pest details can disappear easily
160×160	25,600	76,800 bytes	Low memory cost	Used in early lightweight trials, but detail loss was still important
192×192	36,864	110,592 bytes	Medium memory cost	Better detail preservation for SqueezeNet-inspired trials
224×224	50,176	150,528 bytes	Higher memory cost but better visual detail	Selected for final STFD-MobileNetV1 model
300×300	90,000	270,000 bytes	High memory and computation cost	Used only for EfficientNetB3 software reference experiments

Several preprocessing approaches were tested during the project. These included standard resizing, bilinear interpolation, area interpolation, sharpening after resizing, and color-aware or disease-aware preprocessing trials. These experiments were useful for understanding the effect of resizing and symptom visibility.

Table 5.4 summarizes the preprocessing approaches.

Table 5.4 Preprocessing approaches tested during the project

Step	Description
Standard resize	Baseline image resizing
Bilinear interpolation	Smooth resizing for model input
Area interpolation	Downscaling method tested for preserving small details
Sharpening after resize	Tested to make small symptoms more visible
Color-aware / disease-aware trials	Tested to protect disease-related color regions
Input size trials	Tested 128×128, 160×160, 192×192, 224×224, and 300×300

These preprocessing experiments showed that changing only the resizing method was not enough to improve the results consistently. Some methods made disease colors or edges more visible, but this did not always improve test accuracy. In some cases, stronger preprocessing could also change the natural image appearance and create artificial patterns.

For the final ST FD-MobileNetV1 model, the input image size was 224×224×3. Images were used in RGB format and normalized according to the model preprocessing pipeline.

Figure 5.7 shows a comparison of different resized versions of the same leaf image.

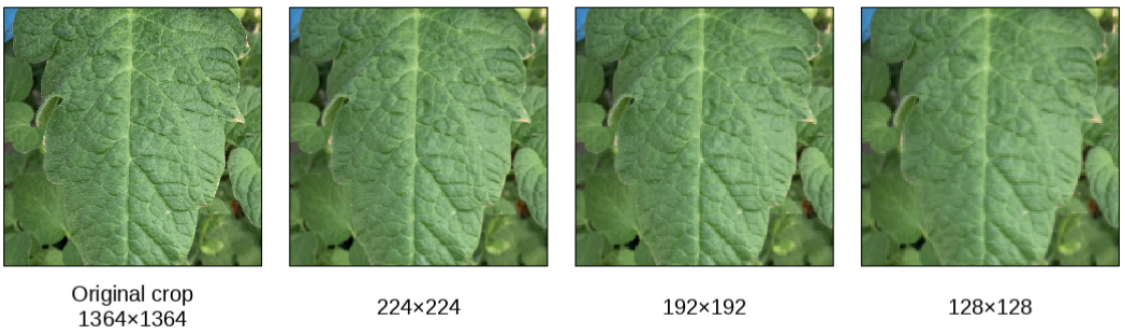


Figure 5.7 Comparison of original, 224×224, 192×192, and 128×128 versions of the same nutdef_leaf image

5.7. Data Augmentation

Data augmentation was applied only during training. The purpose of augmentation was to increase model robustness and reduce overfitting. Since the dataset was collected under real greenhouse conditions, augmentation helped the model become less sensitive to small changes in image position, orientation, and lighting [2] , [3].

The augmentation methods included operations such as:

● horizontal flipping,

● small rotation,

● small translation,

● contrast changes,

● brightness changes,

● light color changes,

● selected stronger augmentation trials in EfficientNetB3 experiments.

Very aggressive augmentation was avoided for the final embedded model because disease or pest symptoms may occupy small regions of the image. Strong zooming or cropping can remove these important regions and make the label less meaningful.

Augmentation experiments were especially useful in the EfficientNetB3 trials, where stronger augmentation methods were tested to understand the upper performance limit of the dataset. However, the final embedded model was selected according to both accuracy and deployment feasibility.

5.8. Model Development Experiments

Several model development experiments were performed during the project. The model design process was iterative. After each experiment, the result was examined and the next experiment was planned according to the observed limitations. The changes were not limited to the model architecture. Different class structures, image sizes, preprocessing methods, interpolation methods, augmentation settings, and training strategies were also tested.

The aim of these experiments was not only to improve accuracy, but also to find a model that could be used in an embedded system. Therefore, model size, input size, quantization suitability, and STM32 deployment feasibility were considered together with test accuracy.

The early experiments included simple CNN and compact CNN trials. These experiments were useful for understanding the difficulty of the collected greenhouse dataset. One of the motivations for these trials was the compact CNN approach in the literature, where smaller CNN models can sometimes perform well for tomato disease classification. However, these early compact CNN trials did not provide sufficient performance for final use on the collected real greenhouse dataset.

ResNet-based models were also tested as early baseline models. These experiments helped to understand whether the dataset contained learnable visual patterns. However, ResNet-based models were not selected for the final embedded system because they were heavier than the target STM32 deployment requirements.

After the early baseline experiments, STM32 AI Model Zoo based ST FD-MobileNetV1 models were tested. The first ST FD-MobileNetV1 trials used a wider class structure with front-side and back-side leaf classes. One early experiment used nine classes, and a later experiment used seven classes. These experiments showed that ST FD-MobileNetV1 was a suitable model family for the project. However, the inclusion of back-side leaf images and less reliable classes increased visual variation and made the classification problem more difficult.

During the project, class selection was updated several times. Earlier experiments included more detailed class structures. Later, the dataset was simplified and cleaned. The final dataset was restricted to five front-side leaf classes: flies_leaf, healthy_leaf, nutdef_leaf,

trips_leaf, and tuta_leaf. This final class structure was selected because it was more consistent with practical greenhouse inspection and gave a more reliable classification task.

EfficientNetB3 was tested as a high-capacity software reference model. This model was not intended for STM32 deployment. Its purpose was to estimate the upper performance limit of the cleaned dataset when a stronger model was used. These experiments helped to show that the final dataset contained learnable visual patterns.

SqueezeNet-like lightweight CNN models were tested as custom embedded candidates. These models used fire-module-like blocks to reduce the number of parameters while keeping useful feature extraction capability. Different input sizes and preprocessing methods were tested with these models. The SqueezeNet-like experiments showed that a small custom CNN could give reasonable performance for tomato leaf classification.

A knowledge distillation experiment was also tested during the project. In this experiment, EfficientNetB3 was used as a stronger teacher model, and a SqueezeNet-like lightweight CNN was used as the student model. The aim was to transfer useful information from the high-capacity teacher model to the smaller embedded candidate. This experiment was useful for understanding an alternative training strategy. However, it was not selected as the final approach because the final ST FD-MobileNetV1 model provided a better balance between accuracy, quantization, and STM32 deployment suitability.

Input image size was another important part of model development. Smaller input sizes such as 128×128 and 160×160 reduced memory and computation cost, but they caused loss of small disease and pest details. Larger input sizes such as 224×224 and 300×300 preserved more visual information but increased computational cost. For the final ST FD-MobileNetV1 model, 224×224 was selected as the input size.

Different preprocessing and augmentation strategies were also tested. These included bilinear resizing, area interpolation, sharpening after resize, color-aware preprocessing trials, brightness and contrast changes, rotation, translation, and stronger augmentation methods in some EfficientNet experiments. These trials showed that preprocessing and augmentation can support training, but dataset quality and class consistency were more important for final performance.

Table 5.5 summarizes the main model development stages.

Table 5.5 Main model development stages

Stage	Model / Experiment	Class Setup	Main Purpose	Design Outcome
Early CNN trials	Simple CNN / compact CNN	Early dataset versions	Test small CNN structures	Not sufficient for final use
Early high-capacity baseline	ResNet-based models	Early dataset versions	Understand dataset learnability	Useful baseline, but heavy for STM32
STFD early trial	ST FD-MobileNetV1	9 classes	Test STM32 Model Zoo model on wider class setup	Good early result, but class setup was less consistent
STFD improved trial	ST FD-MobileNetV1	7 classes	Reduce unreliable classes and test STFD again	Improved class setup
Class selection update	Dataset redesign	5 front-side classes	Improve consistency and practical relevance	Final front-side-only class setup selected
EfficientNetB3	High-capacity CNN	5 front-side classes	Estimate upper software-level performance	Highest software reference result
SqueezeNet-like CNN	Fire-module-like lightweight CNN	5 front-side classes	Test custom embedded candidate	Reasonable lightweight result
Knowledge distillation	EfficientNetB3 teacher + SqueezeNet-like student	5 front-side classes	Improve small model using teacher outputs	Tested as alternative training strategy, not selected as final
STFD final model	ST FD-MobileNetV1	5 front-side classes	Select final STM32-compatible model	Selected as final embedded model
STM32 test	Quantized ST FD-MobileNetV1	5 front-side classes	Test deployed model on STM32	Final embedded test

The final model was selected as ST FD-MobileNetV1 from the STM32 AI Model Zoo. This model was preferred because it gave a good balance between classification accuracy and embedded deployment suitability. It also had a more direct deployment path for STM32 compared with custom high-capacity models.

5.9. SqueezeNet-Inspired Lightweight CNN Experiments

Before selecting the final ST FD-MobileNetV1 model, SqueezeNet-inspired lightweight CNN models were tested as custom embedded candidates. These models were not exact copies of the original SqueezeNet architecture. Therefore, they are described as SqueezeNet-inspired CNN models [4].

The main idea of these models was to reduce the number of parameters while keeping useful feature extraction capability. The model used fire-module-inspired blocks. In these blocks, a 1×1 squeeze convolution is followed by expand branches. This structure helps reduce model size and makes the model more suitable for embedded systems.

The SqueezeNet-inspired experiments were not performed with only one fixed setting. Different class structures, input image sizes, preprocessing methods, and training strategies were tested. Earlier experiments used six leaf-based classes with both front-side and back-side leaf images. Later experiments used the final five front-side leaf classes.

Input image size was one of the important design variables. The first SqueezeNet-inspired experiments used 160×160 input images. This reduced memory usage, but small disease and pest details were harder to preserve. Later, 192×192 input images were tested to keep more visual information while still keeping the model lightweight.

Several preprocessing methods were also tested with SqueezeNet-inspired models. Standard resizing, bilinear interpolation, area interpolation, and area interpolation with sharpening were evaluated. These trials were made to understand whether small disease and pest symptoms could be preserved better after resizing. However, preprocessing changes alone did not provide a consistent improvement.

A knowledge distillation experiment was also tested for the SqueezeNet-inspired model. In this experiment, EfficientNetB3 was used as a stronger teacher model, and the SqueezeNet-inspired CNN was used as the smaller student model. The aim was to transfer useful information from the high-capacity teacher model to the lightweight student model. This experiment was useful because it tested an advanced training strategy for improving a small embedded candidate. However, it was not selected as the final model path because the final ST FD-MobileNetV1 model provided a better balance between accuracy, quantization, and STM32 deployment suitability.

Table 5.6 summarizes the main SqueezeNet-inspired experiment stages.

Table 5.6 SqueezeNet-inspired experiment stages

Trial	Class Setup	Input Size	Method	Main Result / Observation
Early SqueezeNet-inspired trial	6 classes	160×160	Standard resize	Initial lightweight result, but accuracy was limited
Improved 160×160 trial	6 classes	160×160	Standard resize and cleaner training flow	Better than the first trial, but small details were still difficult
Larger-input trial	6 classes	192×192	Bilinear resize	Preserved more visual detail and improved the lightweight candidate
Area interpolation trial	6 classes	192×192	Area interpolation	Tested for better downscaling, but no clear consistent improvement was obtained
Area + sharpening trial	6 classes	192×192	Area interpolation and sharpening	Tested to make small symptoms more visible, but not selected for final use
Knowledge distillation trial	7 classes	192×192	EfficientNetB3 teacher and SqueezeNet-inspired student	Tested as an alternative training strategy, but not selected as final
Final front-side trial	5 classes	192×192	Selected preprocessing	Best SqueezeNet-inspired candidate, but lower than final ST FD-MobileNetV1

The SqueezeNet-inspired experiments showed that a small custom CNN can work for the tomato leaf classification task. They also showed that class structure and input image size strongly affect the result. However, the final ST FD-MobileNetV1 model was preferred because it gave better accuracy and had a more direct STM32 deployment path.

5.10. EfficientNetB3 Upper-Bound Experiments

EfficientNetB3 was tested as a high-capacity software reference model during different stages of the project. These experiments were not intended for final STM32 deployment because EfficientNetB3 is larger and computationally heavier than the lightweight embedded candidates. Instead, EfficientNetB3 was used to understand the performance potential of different dataset versions.

The earlier EfficientNetB3 experiments were performed on wider dataset versions. Some trials used 9-class settings with both front-side and back-side leaf images. Other trials used 6-class and 7-class leaf-based settings. These experiments helped to observe how class selection, dataset cleaning, and class imbalance affected the classification performance.

During these experiments, different training strategies were also tested. These included standard training, stronger augmentation, focal-loss based trials, progressive fine-tuning, MixUp/CutMix style augmentation, and EMA-based training. These trials were useful for understanding whether the accuracy could be improved by changing the training strategy.

After the final dataset decision, one EfficientNetB3 experiment was performed on the five-class front-side leaf dataset. This final EfficientNetB3 experiment was used as the upper-bound software reference for the cleaned final dataset. It showed the possible performance level of the dataset when a high-capacity model was used.

Table 5.7 summarizes the EfficientNetB3 experiment stages.

Table 5.7 EfficientNetB3 experiment stages

Trial Group	Class Setup	Input Size	Main Purpose	Observation
Early EfficientNetB3 trials	9 classes	300×300	Test high-capacity model on wider dataset	Front-side and back-side classes increased difficulty
Six-class EfficientNetB3 trials	6 classes	300×300	Test reduced class setup without tuta_leaf	More consistent than wider setups, but less practical without tuta_leaf
Seven-class EfficientNetB3 trials	7 classes	300×300	Include tuta_leaf while still using back-side classes	Useful intermediate dataset version
Focal / progressive trials	6 classes	300×300	Improve difficult class performance	Tested as training improvement strategy
MixUp / CutMix / EMA trials	7 classes and 5 classes	300×300	Improve generalization and estimate dataset potential	Gave the strongest EfficientNetB3 results
Final EfficientNetB3 reference	5 front-side classes	300×300	Estimate upper-bound performance of final dataset	Highest software-level reference result, not embedded-friendly

These experiments showed that the cleaned dataset contained learnable visual patterns. They also showed that dataset consistency and class selection had a strong effect on performance. However, EfficientNetB3 was not selected as the final model because the main objective was embedded deployment on STM32.

5.11. Final ST FD-MobileNetV1 Model

The final embedded model was based on ST FD-MobileNetV1 from the STM32 AI Model Zoo. This model was selected because it is designed for embedded AI applications and is more suitable for STM32 deployment than larger CNN models [5].

ST FD-MobileNetV1 was not tested only once. Several ST FD-MobileNetV1 experiments were performed during the project with different class settings and evaluation purposes. The earlier ST FD-MobileNetV1 experiment used nine classes, including both front-side and back-side leaf images. A later ST FD-MobileNetV1 experiment used seven leaf-based classes. These experiments were useful because they showed that the STM32 Model Zoo model could work on the tomato leaf dataset. However, the back-side classes increased visual variation and made the dataset less consistent.

After the final class selection decision, the ST FD-MobileNetV1 model was trained again using only five front-side leaf classes. This version was more consistent with the final dataset design. Additional runs were also performed to create cleaner logs, classification reports, and confusion matrix outputs for evaluation.

Table 5.8 summarizes the ST FD-MobileNetV1 development stages.

Table 5.8 ST FD-MobileNetV1 experiment stages

Trial	Class Setup	Input Size	Purpose	Observation
ST FD-MobileNetV1 early trial	9 classes	224×224	Test STM32 Model Zoo model with a wider class setup	Useful early ST model trial, but class setup was less consistent
ST FD-MobileNetV1 improved trial	7 classes	224×224	Test STFD after reducing unreliable classes	Improved structure, but back-side variation remained
ST FD-MobileNetV1 final training	5 front-side leaf classes	224×224	Train the model with the final front-side-only dataset	Selected as final model
ST FD-MobileNetV1 detailed evaluation	5 front-side leaf classes	224×224	Generate cleaner logs, reports, and confusion matrix outputs	Used for final reporting and analysis
STM32 SD card test	5 front-side leaf classes	224×224	Test the deployed quantized model on STM32	Used as the final embedded test

The final model used five selected front-side tomato leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf. The model input size was selected as 224×224×3. During evaluation and deployment, the same class order was used in the Python test code, TensorFlow Lite model, SD card folders, and STM32 label list. This was necessary to avoid label mismatch during embedded testing.

Table 5.8 shows the final ST FD-MobileNetV1 configuration.

Table 5.9 Configuration of the final ST FD-MobileNetV1 model

Parameter	Value
Model	ST FD-MobileNetV1
Source	STM32 AI Model Zoo
Number of classes	5
Input size	224×224×3
Final classes	flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, tuta_leaf
Split	70/15/15
Deployment target	STM32F746G Discovery
Test input method	SD card binary files

5.12. Model Evaluation in Software Environment

After training, the selected models were evaluated on the test set. The test set was not used during training. This made the test result a better indicator of generalization.

The evaluation included:

- test accuracy,
- test loss,
- confusion matrix,
- precision,
- recall,
- F1-score,
- per-class support.

The confusion matrix was used to understand which classes were confused with each other. This was important because some classes were visually similar. For example, pest-related leaf classes could be harder to separate than healthy leaves when insects were small or symptoms were weak.

Software-level evaluation was performed before model conversion. This allowed comparison between the original model, the quantized model, and the final STM32 result.

5.13. TensorFlow Lite Conversion and Quantization

After training, the selected model was converted to TensorFlow Lite format. Quantization was applied to reduce model size and make the model more suitable for microcontroller deployment.

A representative dataset was used during quantization. This helped the converter estimate activation ranges. The quantized model used integer-based input tensors suitable for embedded inference.

After conversion, the quantized model was tested again in the software environment. This step was important because quantization can reduce accuracy. The original model and the quantized model were compared before moving to STM32 testing.

Figure 5.8 shows the simplified quantized model evaluation process.

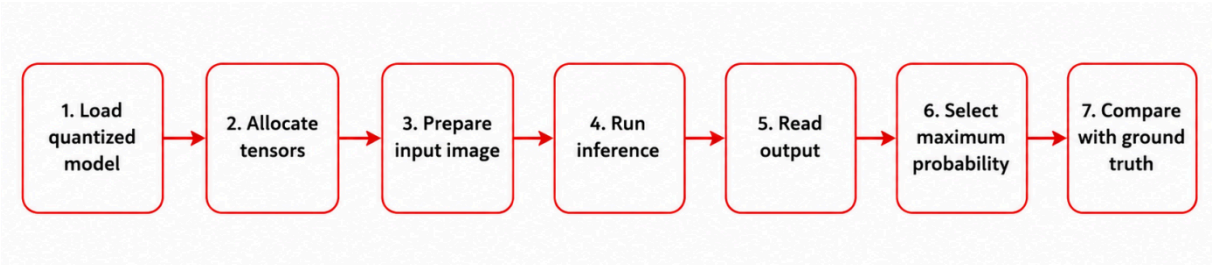


Figure 5.8 The simplified quantized model evaluation process

5.14. Test Image Export for STM32

To test the model on STM32, test images were converted into binary files. These binary files contained the image data in the same format expected by the quantized model input tensor.

For each test image:

1. the image was loaded,
2. it was resized to 224×224 pixels,
3. it was converted into an RGB array,
4. it was quantized according to the model input format,
5. it was saved as a .bin file.

The binary test files were organized into class folders on the SD card. This made it possible to test all images class by class on the STM32 board.

The class folder order was kept consistent with the model output order. This step was important because wrong folder or label order could produce incorrect per-class results.

5.15. STM32 Deployment

The embedded deployment was done on the STM32F746G Discovery platform. The trained and quantized model was integrated into the STM32 project using STM32 AI tools. The embedded project was developed using STM32CubeMX and STM32CubeIDE.

Figure 5.9 shows the STM32 inference pipeline.

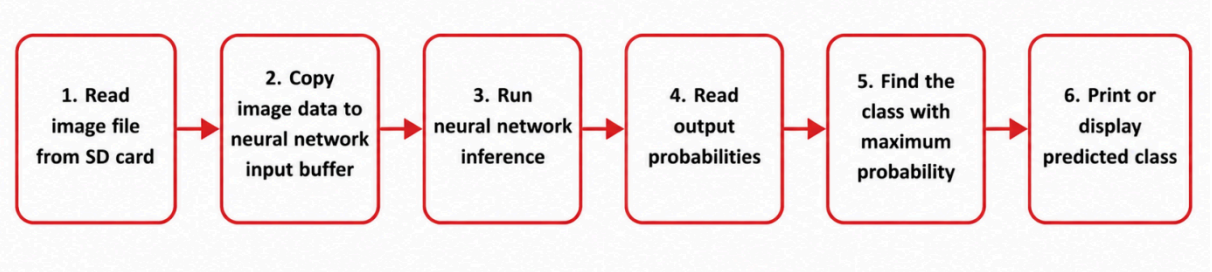


Figure 5.9 STM32 inference pipeline

In earlier tests, a single test image was converted into a C header file and embedded directly into the code. This verified that the STM32 inference pipeline could produce a prediction. Later, the method was improved by using SD card input files because this allowed testing many images without rebuilding the project.

In the final SD card test, the STM32 board mounted the SD card, opened the test folders, read each binary image file, copied the input data to the neural network input buffer, ran inference, read the output probabilities, selected the class with the maximum probability, and compared the predicted class with the folder label.

The output was observed through UART. In single-image tests, the LCD screen was also used to display the predicted class.

5.16. Evaluation Method

The final system was evaluated in three stages.

Table 5.10 Final evaluation stages

Evaluation Stage	Purpose
Software model evaluation	Test the original trained model
Quantized model evaluation	Test the model after conversion and quantization
STM32 SD card evaluation	Test the embedded inference pipeline

The software and quantized model tests were performed using the test set. The STM32 test used preprocessed binary files stored on an SD card. The same class order was used in all stages to avoid label mismatch.

The main evaluation metric was accuracy. However, accuracy alone was not enough because the dataset was not perfectly balanced. Therefore, confusion matrix, precision, recall, and F1-score were also used to analyze class-level performance.

The final STM32 test also counted failed directories and failed files. This was useful for checking not only model accuracy, but also the reliability of the SD card reading and test pipeline.

The final results are presented and discussed in Chapter 6.

6. RESULTS AND DISCUSSION

This chapter presents the experimental results of the project. The results are discussed in terms of dataset quality, model development, preprocessing experiments, software-level accuracy, quantization, and STM32 deployment.

The main goal of the project was not only to obtain high accuracy in a software environment. The final model also had to be suitable for STM32 deployment and had to work with images collected from real greenhouse conditions. Therefore, the results were evaluated by considering classification accuracy, model size, quantization behavior, dataset consistency, and embedded test performance together.

6.1. Dataset Summary

The final dataset was prepared after several rounds of collection, labelling, checking, cleaning, and class selection. The final model used five front-side tomato leaf classes. Fruit, stem, and flower classes were not included in the final classification task because the collected dataset was not suitable for reliable classification of these plant parts. Unhealthy fruit samples were very limited, and no unhealthy stem or flower samples were available.

Back-side leaf images were also removed from the final dataset. Their sample number was lower than front-side leaf images, and their visual appearance was different. This increased the visual variation inside the dataset. Therefore, the final dataset was restricted to front-side leaf images only.

The tuta_leaf class was included in the final dataset because it is practically important for tomato farming, even though it had fewer samples than the other classes.

Table 6.1 shows the final dataset distribution.

Table 6.1 Final front-side leaf dataset distribution

Class	Train	Validation	Test	Total
flies_leaf	1397	300	300	1997
healthy_leaf	3027	649	649	4325
nutdef_leaf	660	141	142	943
trips_leaf	1393	299	299	1991
tuta_leaf	68	15	15	98
Total	6545	1404	1405	9354

The table shows that the dataset was not perfectly balanced. The healthy_leaf class had the highest number of samples, while tuta_leaf had the lowest number of samples. Because of this, the tuta_leaf result should be interpreted carefully. A high score on this class does not necessarily mean that the model fully learned all possible tuta cases, because the test support was small.

Dataset cleaning had an important effect on the project. In the early experiments, unclear images, weak labels, low-quality samples, and inconsistent class structures caused unstable results. After removing ambiguous images and restricting the dataset to front-side leaf images, the classification problem became more consistent. This shows that dataset quality and class selection were as important as model architecture.

Another important point is that the dataset was updated during the project. Early model results and final model results should not be interpreted only as architecture differences. Dataset growth, dataset cleaning, class selection, and error analysis also affected the final performance.

6.2. Model Development Summary

Several model architectures and dataset versions were tested during the project. The model development process was iterative rather than a single-step model selection process. After each experiment, the result was examined and the next experiment was planned according to the observed limitations. Therefore, the final model decision was affected not only by the model architecture, but also by class structure, dataset cleaning, input image size, preprocessing method, augmentation strategy, quantization behaviour, and STM32 deployment feasibility.

In the early stages, compact CNN and ResNet-based models were tested to understand the difficulty of the collected greenhouse dataset. The compact CNN trials showed that very small models were not sufficient for the collected real greenhouse images. The ResNet-based baseline gave a stronger result, but it was not selected as the final model because it was heavier than the target STM32 deployment requirements.

After these early trials, ST FD-MobileNetV1 models from the STM32 AI Model Zoo were tested with wider class structures. The first ST FD-MobileNetV1 trial used a 9-class setup and achieved 86.82% test accuracy. A later 7-class ST FD-MobileNetV1 trial reached 87.40%. These experiments showed that the STM32 Model Zoo model could learn the tomato leaf dataset, but the class structure still included back-side leaf images and visually less consistent categories. Therefore, class selection and dataset cleaning were continued.

SqueezeNet-inspired CNN models were tested as custom lightweight embedded candidates. Different class numbers, input image sizes, and preprocessing methods were tested in these experiments. The final SqueezeNet-inspired model reached 88.29% software accuracy and 88.36% quantized accuracy on the final front-side dataset. This showed that a small custom CNN could work for the problem, but its result was still lower than the final ST FD-MobileNetV1 model.

EfficientNetB3 was tested as a high-capacity software reference model. Earlier EfficientNetB3 trials were performed on wider dataset versions, while the final EfficientNetB3 reference was trained on the five-class front-side leaf dataset. This final EfficientNetB3 experiment reached 96.72% test accuracy. However, EfficientNetB3 was not selected as the final embedded

model because the main objective was not only software accuracy, but also STM32 deployment.

After the dataset was restricted to five front-side leaf classes, the final ST FD-MobileNetV1 model reached 91.25% software test accuracy. The same final model was then deployed and tested on the STM32F746G board using SD card input images, where it reached 90.81% embedded accuracy. This result showed that ST FD-MobileNetV1 provided the best balance between classification accuracy and embedded deployment suitability.

Table 6.2 summarizes the main model development experiments that affected the final model decision.

Table 6.2 Summary of selected model development experiments

Model / Experiment	Setup	Result	Purpose / Decision
Early compact CNN	7 classes, 192/224	76.50%	Early small CNN baseline; not sufficient for final use
ResNet-based baseline	6 classes, 224×224	88.58%	Stronger software baseline, but heavy for STM32
ST FD-MobileNetV1 early trial	9 classes, 224×224	86.82%	First STM32 Model Zoo trial with wider class setup
ST FD-MobileNetV1 improved trial	7 classes, 224×224	87.40%	Reduced class setup, but back-side variation remained
SqueezeNet-inspired CNN	5/6 classes, 192×192	88.36%	Best custom lightweight CNN candidate
EfficientNetB3	5 classes, 300×300	96.72%	Upper-bound software reference
ST FD-MobileNetV1 final	5 classes, 224×224	91.25%	Final selected software model
ST FD-MobileNetV1 on STM32	5 classes, 224×224	90.81%	Final embedded test

The results in Table 6.2 show that the highest software-level accuracy was obtained with EfficientNetB3. However, this model was not selected for the final embedded implementation because it was larger and computationally heavier. The SqueezeNet-inspired

model was smaller and more embedded-friendly, but its final accuracy was lower than ST FD-MobileNetV1.

The ST FD-MobileNetV1 experiments show the effect of dataset and class structure. The 9-class and 7-class trials produced useful results, but they still included back-side leaf images and less consistent visual variations. After the final five-class front-side dataset was prepared, the ST FD-MobileNetV1 result improved and became more suitable for the final system.

Therefore, the final model decision was made by considering both accuracy and deployment feasibility. EfficientNetB3 was used as the upper-bound reference, SqueezeNet-inspired CNN was used as a custom lightweight candidate, and ST FD-MobileNetV1 was selected as the final embedded model because it provided the best balance between software accuracy, quantization behaviour, and STM32 test performance.

6.3. Preprocessing Experiments

Image preprocessing was an important part of the project because small disease marks and insects can become difficult to see after resizing. Several preprocessing methods were tested to understand whether symptom visibility could be improved before training.

Table 6.3 summarizes the main preprocessing and augmentation-related trials.

Table 6.3 Summary of preprocessing and augmentation experiments

Preprocessing Method	Purpose	Observation
Standard resize	Baseline preprocessing	Stable and simple
Bilinear interpolation	Smooth image resizing	Used in several baseline trials
Area interpolation	Improve downscaling quality for small details	No clear and consistent improvement
Area resize + sharpening	Increase visibility of small pest or disease marks	Did not provide a meaningful gain in the final results
Color-aware / disease-aware trials	Preserve disease-related color regions	Tested as an alternative idea, but not selected for final model
Input size changes	Compare detail preservation and memory cost	Smaller inputs reduced cost but caused loss of small pest details
Standard augmentation	Improve robustness to small position and lighting changes	Useful for training stability
Strong augmentation / training strategies	Improve generalization in high-capacity models	MixUp, CutMix, and EMA improved EfficientNetB3 experiments

The results in Table 6.3 show that basic preprocessing changes alone did not solve the classification problem. Area interpolation was tested because it can be more suitable for downscaling images, and sharpening was tested to make small pest marks more visible. However, these resize-based methods did not provide a clear and consistent improvement in the final results.

This result is important because the main limitation was not only the resizing algorithm. If the original symptom was too small, unclear, or visually weak, interpolation or sharpening could not fully recover the missing information after resizing. In some cases, stronger preprocessing also changed the natural appearance of the leaf and did not improve the model result.

However, this does not mean that all augmentation or training improvements were ineffective. In the EfficientNetB3 experiments, stronger training strategies such as MixUp, CutMix, and EMA-based training improved the software-level performance significantly. These methods were more effective than simple resizing changes because they improved model generalization during training rather than only modifying image appearance before training.

The effect of these methods also depends on the model architecture. EfficientNetB3 is a high-capacity model, so it can benefit more from stronger augmentation and regularization strategies. These methods can reduce overfitting and help the model learn smoother decision boundaries. Therefore, applying MixUp, CutMix, and EMA to EfficientNetB3 was useful for estimating the upper software-level performance of the dataset.

In theory, these stronger training strategies could also be tested with SqueezeNet-inspired CNNs or ST FD-MobileNetV1 models. However, their effect on smaller embedded-oriented models is not guaranteed. Lightweight models have lower learning capacity, and aggressive augmentation can sometimes make the learning problem harder. This is especially important in this project because some class differences depend on small visual symptoms such as flies, trips marks, or early pest damage. If these small regions are mixed, cropped, or visually weakened too much, the label information may become harder for a small model to learn.

For this reason, stronger augmentation strategies were mainly used with EfficientNetB3 as an upper-bound software reference. The lightweight model experiments focused more on embedded suitability, quantization behavior, input size, and STM32 deployment feasibility. The final system therefore used a stable preprocessing and augmentation pipeline instead of an aggressive image enhancement method.

Overall, the experiments showed that basic resize-based preprocessing alone did not create a consistent gain, while stronger augmentation strategies were useful especially for high-capacity software reference models. Testing MixUp, CutMix, EMA, and similar methods more systematically on lightweight embedded models can be considered as future work.

6.4. EfficientNetB3 Upper-Bound Results

EfficientNetB3 was tested as a high-capacity software reference model during the project. These experiments were not intended for final STM32 deployment because EfficientNetB3 is larger and computationally heavier than the lightweight embedded candidates. Instead, EfficientNetB3 was used to understand the possible software-level performance of different dataset versions.

EfficientNetB3 was tested on several dataset versions. Earlier trials used wider class structures, including 9-class, 7-class, and 6-class settings. These experiments helped to observe how class selection, back-side leaf images, dataset cleaning, and class imbalance affected the result. Since the dataset was still changing during the project, these results should not be interpreted only as architecture differences. They were also affected by dataset growth and dataset cleaning.

The EfficientNetB3 experiments also showed the importance of training strategy. In some trials, the dataset version was kept similar while the training strategy was changed. Stronger strategies such as MixUp, CutMix, EMA-based training, stronger augmentation, and progressive fine-tuning improved the EfficientNetB3 results. This showed that the model architecture was not the only factor affecting accuracy. Training strategy also had an important effect, especially for high-capacity models.

The final EfficientNetB3 reference experiment was performed on the five-class front-side leaf dataset. This experiment reached 96.72% test accuracy, which was the highest software-level result obtained in the project. This result showed that the cleaned front-side dataset contained learnable visual patterns and that high accuracy was possible when a larger model was used.

Table 6.4 summarizes the EfficientNetB3 results.

Table 6.4 Summary of EfficientNetB3 results

Experiment	Classes	Input Size	Main Result	Role
Early EfficientNetB3 trials	9	300×300	About 87–89%	Wider dataset reference
Reduced class EfficientNetB3 trials	6 / 7	300×300	About 89–95%	Dataset and augmentation analysis
EfficientNetB3 with stronger training strategies	7 / 5	300×300	Improved software-level result	MixUp, CutMix, EMA, and stronger augmentation trials
Final EfficientNetB3 reference	5	300×300	96.72%	Upper-bound software reference

The results in Table 6.4 show that EfficientNetB3 benefited from stronger training strategies and the final cleaned dataset. However, the model was not selected as the final embedded model because the target platform was a microcontroller. A model with the highest software accuracy is not always the best embedded model. The final choice must also satisfy model size, memory usage, quantization behaviour, and STM32 deployment requirements.

For this reason, EfficientNetB3 was used as an upper-bound experiment. It showed the performance potential of the final dataset, while ST FD-MobileNetV1 was selected as the final embedded model because it provided a better balance between accuracy and deployment suitability.

6.5. SqueezeNet-Inspired Lightweight CNN Results

SqueezeNet-inspired CNN models were tested as custom lightweight embedded candidates before selecting the final ST FD-MobileNetV1 model. These models used fire-module-inspired blocks and were designed to reduce the number of parameters compared with larger CNN architectures. The main purpose of these experiments was to test whether a custom small CNN could provide acceptable accuracy while remaining suitable for embedded deployment.

The SqueezeNet-inspired experiments were not performed with only one fixed setting. Different class structures, input image sizes, preprocessing methods, and training strategies were tested. Earlier experiments used six leaf-based classes, while the final SqueezeNet-inspired experiment used the five-class front-side dataset.

Input image size was one of the main design variables. The first SqueezeNet-inspired trials used 160×160 input images. This reduced memory and computation cost, but small pest symptoms were harder to preserve. Later, 192×192 input images were tested to keep more visual detail while still keeping the model lightweight. The 192×192 trials gave better results than the earlier 160×160 trials, showing that input resolution was important for small symptom visibility.

Several preprocessing methods were also tested with the SqueezeNet-inspired model. Area interpolation and area interpolation with sharpening were tested because they could potentially preserve small pest or disease details better after resizing. However, these preprocessing changes did not provide a clear and consistent improvement. This supported the observation that preprocessing alone could not solve problems caused by weak or ambiguous visual symptoms.

A knowledge distillation experiment was also tested for the SqueezeNet-inspired model. In this experiment, EfficientNetB3 was used as a teacher model, and the SqueezeNet-inspired CNN was used as the student model. The aim was to improve the smaller model by transferring useful information from the stronger teacher model. However, this experiment did not become the final model path because the final ST FD-MobileNetV1 model provided a better balance between accuracy, quantization, and STM32 deployment suitability.

Table 6.5 summarizes the main SqueezeNet-inspired results.

Table 6.5 Summary of SqueezeNet-inspired CNN results

Trial	Classes	Input Size	Accuracy	Quantized Accuracy	Role
Early SqueezeNet-inspired trial	6	160×160	82.20%	-	First custom lightweight trial
Improved 160×160 trial	6	160×160	85.35%	85.61%	Improved lightweight trial
192×192 trial	6	192×192	85.61%	85.77%	Larger input trial
Knowledge distillation trial	7	192×192	-	84.93%	Teacher-student trial
Final front-side SqueezeNet-inspired trial	5	192×192	88.29%	88.36%	Best SqueezeNet-inspired result

The results in Table 6.5 show that the SqueezeNet-inspired model improved as the input size and dataset structure became more suitable. The early 160×160 trial had limited accuracy, while the 192×192 version preserved more visual detail and gave better results. The final five-class front-side version reached 88.29% software accuracy and 88.36% quantized

accuracy.

The quantized accuracy being close to the software accuracy was a positive result. It showed that the SqueezeNet-inspired model could be quantized without a large performance loss. This made it a reasonable lightweight embedded candidate.

However, the SqueezeNet-inspired model was not selected as the final system. Although it was small and embedded-friendly, its final accuracy remained lower than the final ST FD-MobileNetV1 model. The ST FD-MobileNetV1 model also had a more direct deployment path through the STM32 AI Model Zoo. Therefore, the SqueezeNet-inspired model was useful as a custom lightweight experiment, but ST FD-MobileNetV1 was selected as the final embedded model.

6.6. Final ST FD-MobileNetV1 Software Result

ST FD-MobileNetV1 was selected as the final embedded model because it provided the best balance between software accuracy, quantization behavior, and STM32 deployment suitability. Unlike EfficientNetB3, it was designed for embedded AI applications. Unlike the custom SqueezeNet-inspired model, it also had a more direct deployment path through the STM32 AI Model Zoo and STM32 AI tools.

ST FD-MobileNetV1 was not tested only once during the project. The first ST FD-MobileNetV1 experiment used a wider 9-class setup, including both front-side and back-side leaf images. This experiment reached 86.82% test accuracy. Although this result showed that the STM32 Model Zoo model could learn the tomato leaf dataset, the class structure was not ideal because it still included visually different back-side leaf classes.

A later ST FD-MobileNetV1 experiment used a reduced 7-class setup and reached 87.40% test accuracy. This showed a small improvement, but the dataset still contained front-side and back-side variation. Therefore, the class structure was further simplified and cleaned.

After the final dataset decision, ST FD-MobileNetV1 was trained again using only five front-side leaf classes. This final class structure was more consistent with practical greenhouse inspection because disease and pest symptoms are generally easier to observe on the upper/front surface of the leaf. The final ST FD-MobileNetV1 software model reached 91.25% test accuracy.

Table 6.6 summarizes the ST FD-MobileNetV1 development results.

Table 6.6 Summary of ST FD-MobileNetV1 results

Experiment	Classes	Input Size	Main Result	Role
ST FD-MobileNetV1 early trial	9	224×224	86.82%	Early STM32 Model Zoo trial
ST FD-MobileNetV1 improved trial	7	224×224	87.40%	Reduced class setup trial
ST FD-MobileNetV1 five-class trial	5	224×224	90.96% float, 91.10% quantized	Final dataset training trial
ST FD-MobileNetV1 final software test	5	224×224	91.25%	Final selected software model
ST FD-MobileNetV1 STM32 test	5	224×224	90.81%	Final embedded test

The results in Table 6.6 show that the improvement was not caused only by changing the model architecture. The same model family produced better results as the dataset became cleaner and the class structure became more consistent. Removing back-side leaf images reduced visual variation, and focusing on five front-side classes made the classification task more practical and reliable.

The final software accuracy of 91.25% was lower than the EfficientNetB3 upper-bound result, but it was higher than the final SqueezeNet-inspired result. More importantly, ST FD-MobileNetV1 was more suitable for STM32 deployment than EfficientNetB3. It also produced a stable embedded result on the STM32 board, reaching 90.81% accuracy in the SD card test.

Therefore, ST FD-MobileNetV1 was selected as the final model. This decision was based on the complete project objective: achieving acceptable classification accuracy on real greenhouse images while also keeping the model suitable for quantization and STM32 deployment.

6.7. Final Per-Class Software Results

The final ST FD-MobileNetV1 model was trained using five front-side tomato leaf classes with 224×224 input images. The model was first evaluated in the software environment before STM32 deployment. This step was important because it showed the classification performance of the trained model before embedded testing.

Table 6.7 shows the final software-level result of the selected ST FD-MobileNetV1 model.

Table 6.7 Final ST FD-MobileNetV1 software result

Metric	Value
Model	ST FD-MobileNetV1
Number of classes	5
Input size	224×224
Test images	1405
Test loss	0.2434
Test accuracy	91.25%
Macro F1-score	90.67%
Weighted F1-score	91.16%

The final software accuracy was 91.25%. This result shows that the selected ST FD-MobileNetV1 model performed well on the cleaned five-class front-side leaf dataset. The weighted F1-score was close to the test accuracy, which indicates that the overall model performance was consistent with the class-level results.

However, macro F1-score should be interpreted together with the class support values. The dataset was not perfectly balanced, and the tuta_leaf class had only 15 test samples. Therefore, a high score for tuta_leaf does not necessarily mean that the model learned all possible tuta_leaf variations. More tuta_leaf samples are needed for a stronger evaluation of this class.

Table 6.8 shows the per-class software results of the final ST FD-MobileNetV1 model.

Table 6.8 Per-class software results of the final ST FD-MobileNetV1 model

Class	Precision	Recall	F1-score	Support
flies_leaf	0.8836	0.8100	0.8452	300
healthy_leaf	0.9145	0.9553	0.9344	649
nutdef_leaf	0.9236	0.9366	0.9301	142
trips_leaf	0.9315	0.9097	0.9205	299
tuta_leaf	0.8750	0.9333	0.9032	15
Accuracy	-	-	0.9125	1405
Macro avg	0.9056	0.9090	0.9067	1405
Weighted avg	0.9120	0.9125	0.9116	1405

The weakest recall value was obtained for flies_leaf. The recall of flies_leaf was 0.8100, which means that some true flies_leaf images were classified as other classes. This result is understandable because fly symptoms can be very small and visually weak. If only a few insects are visible on the leaf, the image can look similar to healthy_leaf after resizing to 224×224.

The healthy_leaf class achieved high recall with 0.9553. This means that most healthy_leaf samples were correctly identified. However, the confusion matrix should also be checked because some unhealthy or pest-related images may still be classified as healthy_leaf when the visible symptoms are weak.

The nutdef_leaf and trips_leaf classes also achieved strong F1-scores. The nutdef_leaf class reached 0.9301 F1-score, and the trips_leaf class reached 0.9205 F1-score. These results show that the model could learn visible nutrient deficiency and trips-related visual patterns when the symptoms were clear enough.

The tuta_leaf class reached a high F1-score of 0.9032. However, this result must be interpreted carefully because the test support was only 15 images. Since this class had the lowest number of samples in the dataset, more tuta_leaf images should be collected in future work to evaluate the class more reliably.

Overall, the per-class software results show that the final model performed well on most classes. The main limitation was related to weak or small pest symptoms, especially in the flies_leaf class. This result supports the earlier dataset cleaning observations, where samples with unclear or very weak visual evidence were found to be problematic after resizing.

The software confusion matrix was used to understand which classes were confused with each other by the final ST FD-MobileNetV1 model. Accuracy and F1-score give general performance information, but the confusion matrix shows the actual error pattern between classes.

Table 6.9 shows the software confusion matrix of the final model.

Table 6.9 Software confusion matrix of the final ST FD-MobileNetV1 model

True \ Predicted	flies_leaf	healthy_leaf	nutdef_leaf	trips_leaf	tuta_leaf
flies_leaf	243	39	6	11	1
healthy_leaf	22	620	1	6	0
nutdef_leaf	2	4	133	3	0
trips_leaf	8	14	4	272	1
tuta_leaf	0	1	0	0	14

The largest confusion occurred between flies_leaf and healthy_leaf. In the software test, 39 flies_leaf samples were classified as healthy_leaf. This means that some leaves containing flies were predicted as healthy. This is one of the most important error types because it shows that weak pest evidence can be missed by the model.

This error is understandable when the visual structure of the dataset is considered. In some flies_leaf images, the insects are very small or only a few flies are visible. After resizing the image to 224×224 pixels, these small details can become harder to see. As a result, the leaf can look visually similar to a healthy leaf. This explains why flies_leaf had the lowest recall value in the per-class results.

There was also confusion in the opposite direction. The model classified 22 healthy_leaf samples as flies_leaf. This may happen when small dark spots, shadows, or natural leaf texture are visually similar to small insects. Therefore, the model sometimes interpreted harmless visual details as fly evidence.

Another visible confusion occurred between trips_leaf and healthy_leaf. In the software test, 14 trips_leaf samples were classified as healthy_leaf. Trips-related symptoms can also be weak or visually subtle. If the damage is small or not clearly visible, the model may not separate it from healthy_leaf.

The nutdef_leaf class showed a strong diagonal result with 133 correct predictions out of 142 test samples. This indicates that nutrient deficiency patterns were generally more distinguishable than small pest symptoms. The tuta_leaf class also had 14 correct predictions out of 15 test samples, but this result should be interpreted carefully because the test support was very small.

Overall, the software confusion matrix shows that the final model performed well when the visual symptoms were clear. Most errors occurred when the symptoms were small, weak, or visually similar to another class. This supports the main observation of the project: dataset quality, symptom visibility, and class consistency are critical for real greenhouse image classification.

6.8. Quantization Result

After training, the final ST FD-MobileNetV1 model was converted and quantized before STM32 deployment. Quantization was necessary because the target platform was a microcontroller with limited memory and processing capacity. A model that performs well in the software environment must also remain usable after conversion and deployment.

The original software model reached 91.25% test accuracy on the final five-class test set. After the model was converted and deployed on the STM32F746G board, the embedded SD card test reached 90.81% accuracy. The difference between the software result and the embedded result was small, which shows that the deployed model preserved most of its software-level performance on the target board.

Table 6.10 compares the software and embedded test results.

Table 6.10 Software and embedded test accuracy comparison

Evaluation Type	Platform	Input Method	Accuracy
Original software model	Google Colab / TensorFlow	Test image dataset	91.25%
Quantized model	STM32F746G board	SD card binary image files	90.81%

The result in Table 6.10 shows that the final model was suitable for embedded deployment. The embedded accuracy was close to the original software accuracy, although the STM32 test included the complete embedded inference pipeline, including SD card reading, binary image loading, input buffer preparation, model inference, output reading, and class comparison.

This comparison is important because the project goal was not only to train a model with high software accuracy. The main goal was to build a working embedded AI pipeline. Therefore, the close software and STM32 results show that the selected ST FD-MobileNetV1 model was a suitable final choice for the project.

The small difference between the software and embedded results can be caused by model conversion, quantization, input formatting, or numerical differences between the software environment and the embedded inference environment. However, the difference was acceptable for this project, and the final STM32 result confirmed that the model could run successfully on the target microcontroller platform.

6.9. STM32 SD Card Test Result

The final model was deployed on the STM32F746G board and tested using SD card input images. The test images were converted into binary files and organized into class folders on the SD card. During the test, the STM32 board read each binary image file, copied the image data to the neural network input buffer, ran inference, selected the predicted class, and compared the prediction with the true class folder.

The STM32 test used 224×224×3 input images. Each binary image file had an expected size of 150,528 bytes. The deployed model had five output classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf.

Figure 6.1 shows the UART output of the final STM32 SD card test. This output confirms that the deployed model was tested directly on the STM32F746G board using binary image files stored on the SD card. A total of 1405 test images were processed, and 1276 of them were classified correctly. The final embedded accuracy was 90.81%.

Another important result in Figure 6.1 is that the failed directory and failed file counts were both zero. This shows that the SD card reading, file loading, input buffer preparation, and inference loop worked correctly during the full test. Therefore, the STM32 result does not only represent model accuracy; it also verifies that the embedded test pipeline operated successfully.

```
====================================
        SD TEST FINISHED
====================================
Total files tested : 1405
Correct            : 1276 / 1405
Accuracy           : 90.81 %
Failed dirs        : 0
Failed files       : 0

Per-class results:
00 flies                : 249 / 300  acc=83.00 %
01 healthy              : 614 / 649  acc=94.60 %
02 nutdef               : 133 / 142  acc=93.66 %
03 trips                : 266 / 299  acc=88.96 %
04 tuta                 : 14 / 15  acc=93.33 %

Confusion matrix
Rows=true class, Cols=pred class

      00 01 02 03 04
00:   249 39 6 6 0  | flies
01:   30 614 2 3 0  | healthy
02:   2 4 133 3 0  | nutdef
03:   16 12 4 266 1  | trips
04:   0 1 0 0 14  | tuta

Class index list:
00 -> flies
01 -> healthy
02 -> nutdef
03 -> trips
04 -> tuta

Test done.
```

Figure 6.1 UART output of the final STM32 SD card test

Table 6.11 shows the final STM32 SD card test result.

Table 6.11 Final STM32 SD card test result

Metric	Value
Total files tested	1405
Correct predictions	1276
Accuracy	90.81%
Failed directories	0
Failed files	0
Input size	224×224×3
Number of classes	5
Expected binary file size	150,528 bytes

The result in Table 6.11 shows that the embedded inference pipeline worked successfully. The final STM32 accuracy was close to the software-level accuracy of 91.25%. This is an important result because it shows that the model behavior remained consistent after conversion, deployment, and execution on the STM32 board.

The SD card test was also more meaningful than a single-image demonstration. A single-image test can only show that the model produces an output, but the SD card test evaluated the deployed model on the full test set. Therefore, this test verified both the model performance and the reliability of the embedded testing workflow.

6.10. STM32 Per-Class Results

The STM32 SD card test was also evaluated class by class. This was important because the overall accuracy alone does not show which classes were easier or harder for the embedded model. Since the dataset was not perfectly balanced, per-class results helped to understand the model behavior more clearly.

Table 6.12 shows the per-class STM32 results.

Table 6.12 Per-class STM32 results

Class	Correct / Total	Accuracy
flies_leaf	249 / 300	83.00%
healthy_leaf	614 / 649	94.60%
nutdef_leaf	133 / 142	93.66%
trips_leaf	266 / 299	88.96%
tuta_leaf	14 / 15	93.33%

The STM32 per-class results show that the model performed well for most classes. The highest results were obtained for healthy_leaf, nutdef_leaf, and tuta_leaf. However, the tuta_leaf result should be interpreted carefully because this class had only 15 test images. Therefore, a high accuracy value for tuta_leaf does not guarantee that the model can generalize to all possible tuta_leaf cases.

The lowest STM32 result was obtained for flies_leaf with 83.00% accuracy. This is consistent with the software-level per-class results, where flies_leaf also had the lowest recall. The main reason is that fly evidence can be visually weak. In some images, only a few small flies are visible on the leaf. After resizing to 224×224 pixels, these small details can become harder to detect, and the image may look similar to healthy_leaf.

The trips_leaf class reached 88.96% accuracy. This result is lower than healthy_leaf and nutdef_leaf, but still acceptable. Trips-related symptoms can sometimes be small or visually similar to other pest-related patterns. Therefore, some confusion between trips_leaf, flies_leaf, and healthy_leaf is expected.

The nutdef_leaf class reached 93.66% accuracy. This result suggests that nutrient deficiency symptoms were more visually distinguishable than small insect-related symptoms. Nutrient deficiency usually causes larger color or texture changes on the leaf, which can remain visible after resizing.

Overall, the per-class STM32 results support the software-level analysis. The model performed better when the visual symptoms were clear and larger, while performance decreased when the class depended on small or weak pest symptoms. This shows that symptom visibility was one of the main factors affecting the final embedded performance.

6.11. STM32 Confusion Matrix Discussion

The STM32 confusion matrix was used to analyze the error pattern of the deployed model on the board. This is important because the embedded test result should not be evaluated only with overall accuracy. The confusion matrix shows which classes were confused with each other during the STM32 SD card test.

Figure 6.3 shows the confusion matrix of the STM32 SD card test.

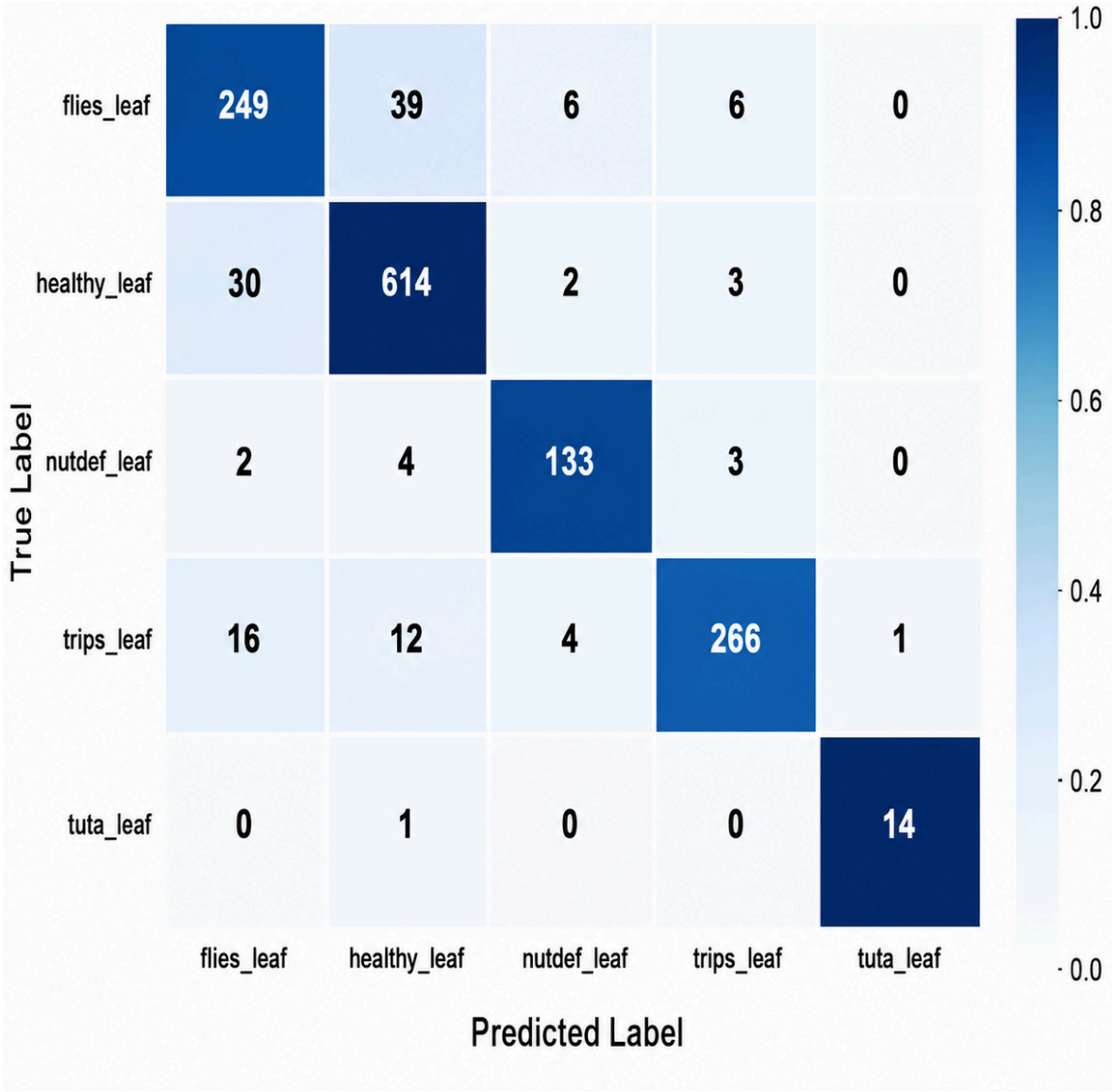


Figure 6.2 Confusion matrix for the STM32 test

The confusion matrix shows that most predictions were located on the diagonal, which means that the model correctly classified most test images. This supports the final embedded accuracy of 90.81%. However, some class pairs had visible confusion.

The most important confusion occurred between flies_leaf and healthy_leaf. Some true flies_leaf images were predicted as healthy_leaf, and some true healthy_leaf images were predicted as flies_leaf. This result is consistent with the software-level confusion matrix and the per-class STM32 results. The main reason is that fly evidence can be very small. If only a few small flies are visible on the leaf, the image can become visually similar to healthy_leaf after resizing to 224×224 pixels.

Another source of confusion was related to trips_leaf. Some trips_leaf images were predicted as flies_leaf or healthy_leaf. This is also expected because pest-related symptoms can be weak and visually similar. In real greenhouse images, lighting, shadows, leaf texture, and camera distance can make these small symptoms harder to separate.

The nutdef_leaf class showed stronger performance compared with the pest-related classes. This is because nutrient deficiency symptoms usually create larger visible changes in color or texture. These changes are easier to preserve after resizing than very small insect details.

The tuta_leaf class had only one wrong prediction in the STM32 test. However, this result should not be overinterpreted because the tuta_leaf test set contained only 15 images. More tuta_leaf data is needed to evaluate this class more reliably.

Overall, the STM32 confusion matrix confirms the main limitation of the final system. The model performs well when the symptom is clear and visible, but errors increase when the symptom is small, weak, or visually similar to another class. This result also supports the dataset cleaning decisions made during the project, especially the removal of unclear samples and weak pest images.

6.12. Discussion of Satisfactory and Unsatisfactory Parts

The final result is satisfactory for the main goal of the project. The system did not only train a model in a software environment, but also deployed and tested the selected model on an STM32 board. The final ST FD-MobileNetV1 model reached 91.25% software test accuracy and 90.81% STM32 SD card test accuracy. This shows that the embedded AI pipeline worked successfully on the selected test set.

One of the most satisfactory parts of the project was the creation of a custom dataset from real greenhouse conditions. The dataset was not taken from a ready public source. Instead, the images were collected manually, labelled using CVAT, checked several times, and cleaned by removing unclear, weakly labelled, and visually ambiguous samples. This dataset preparation process was one of the most important contributions of the project.

Another satisfactory result was the comparison of different model families and dataset versions. EfficientNetB3 was useful as a high-capacity software reference model, SqueezeNet-inspired CNNs were useful as custom lightweight candidates, and ST FD-MobileNetV1 was selected as the final embedded model. This process showed that the final model decision was not based only on a single accuracy value, but on the balance between accuracy, quantization behavior, and STM32 deployment feasibility.

The STM32 SD card test was also successful. A total of 1405 test images were processed on the board, and 1276 images were classified correctly. The failed directory and failed file counts were both zero. This means that the SD card reading, file loading, input buffer preparation, inference loop, and output comparison steps worked correctly during the embedded test.

The most satisfactory parts of the project can be summarized as follows:

● a custom tomato leaf dataset was created from real greenhouse images,

● the dataset was labelled, cleaned, and corrected through an iterative process,

● unclear and visually ambiguous samples were removed or corrected,

● several model architectures and dataset versions were compared,

● the final five-class front-side leaf dataset produced a more consistent classification task,

● the final ST FD-MobileNetV1 model achieved 91.25% software accuracy,

● the deployed STM32 system achieved 90.81% embedded accuracy,

● the SD card based STM32 test was completed with zero failed files,

● the complete workflow from dataset collection to STM32 inference was demonstrated.

However, some limitations remain. The dataset is still not perfectly balanced. The tuta_leaf class had only 15 test images, so its high test result should be interpreted carefully. More tuta_leaf samples are needed before making a stronger conclusion about this class.

Another limitation is the lower performance of the flies_leaf class. In both software and STM32 tests, flies_leaf was one of the most difficult classes. This is mainly because fly evidence can be very small and visually weak. After resizing to 224×224 pixels, some flies_leaf images may look similar to healthy_leaf images.

The system also does not use live camera input in its current form. In this project, the STM32 board used preprocessed binary image files stored on an SD card. This was enough to test the embedded inference pipeline on the full test set, but it is not yet a complete real-time field system. Camera integration should be added in future work.

Another limitation is that the model was tested on the collected greenhouse dataset only. More data from different greenhouses, different lighting conditions, different plant growth stages, and different disease stages would be needed to evaluate the robustness of the system more completely.

In summary, the final system meets the main project objective. It shows that tomato leaf health classification can be performed on a microcontroller-based embedded system using a lightweight model. At the same time, the results show that dataset quality, class balance, symptom visibility, and live camera integration are important issues for future improvement.

7. CONCLUSION

In this project, a microcontroller-based artificial intelligence system was developed for tomato plant health monitoring under real greenhouse conditions. The main goal was to classify tomato leaf images using a lightweight deep learning model and to deploy the selected model on an STM32 microcontroller. The project was not limited to software-level model training. It also included dataset collection, manual labelling, dataset cleaning, model comparison, TensorFlow Lite conversion, quantization, STM32 integration, and SD card based embedded testing.

One of the most important parts of the project was dataset preparation. The images were collected manually from tomato plants in greenhouse conditions over several weeks. The dataset was not taken from a ready and clean public dataset. Therefore, the images included real-world difficulties such as different lighting conditions, different camera angles, shadows, background objects, focus problems, leaf position changes, and different symptom visibility levels. After image collection, the images were labelled using CVAT and reviewed several times.

The dataset also changed and improved during the project. While model experiments were being performed, new images were still being collected, labelled, checked, and cleaned. In addition, wrongly predicted or suspicious test images were reviewed after some experiments. Some images were found to be ambiguous, unclear, or wrongly labelled. These samples were corrected or removed from the dataset. This process showed that dataset quality was as important as model architecture in real greenhouse image classification.

The initial scope of the project included leaves, fruits, stems, and flowers. However, the collected dataset was not suitable for reliable classification of all these plant parts. Unhealthy fruit samples were very limited, and no unhealthy stem or flower samples were available. Therefore, fruit, stem, and flower categories were not used in the final classifier. Back-side leaf images were also removed because their sample number was lower and their visual appearance was different from front-side leaf images. As a result, the final system focused on five front-side leaf classes: flies_leaf, healthy_leaf, nutdef_leaf, trips_leaf, and tuta_leaf.

Several model architectures and training strategies were tested during the project. Early experiments included compact CNN trials, ResNet-based models, and different ST FD-MobileNetV1 trials with wider class structures. EfficientNetB3 was used as a high-capacity software reference model to estimate the upper performance limit of the cleaned dataset. SqueezeNet-inspired lightweight CNN models were tested as custom embedded candidates. A knowledge distillation experiment was also tested by using EfficientNetB3 as a teacher model and a SqueezeNet-inspired model as a student model.

The final embedded model was selected as ST FD-MobileNetV1 from the STM32 AI Model Zoo. This model provided a good balance between classification accuracy, quantization behavior, and STM32 deployment suitability. The final ST FD-MobileNetV1 model achieved 91.25% test accuracy in the software environment. After deployment on the STM32F746G board, the model was tested using preprocessed binary image files stored on an SD card. In the STM32 test, 1405 images were tested, 1276 images were classified correctly, and the final embedded accuracy was 90.81%. The SD card test also had zero failed directories and zero failed files, showing that the embedded test pipeline worked reliably.

The final results show that tomato leaf health classification can be performed on a microcontroller-based embedded system using a lightweight AI model. The close difference between the software accuracy and the STM32 test accuracy shows that the selected model preserved most of its performance after deployment. This confirms that the final ST FD-MobileNetV1 model was suitable for the target embedded system.

However, some limitations remain. The dataset was not perfectly balanced, especially for the tuta_leaf class. The number of tuta_leaf test samples was small, so its result should be interpreted carefully. More tuta_leaf samples are needed for a more reliable evaluation of this class. The flies_leaf class also had lower performance compared with some other classes because fly symptoms can be small and visually weak. After resizing to 224×224 pixels, some flies_leaf images may look similar to healthy_leaf images.

Another limitation is that the final STM32 system used preprocessed binary images stored on an SD card instead of live camera input. This was enough to test the embedded inference pipeline on the full test set, but it is not yet a complete real-time greenhouse monitoring

system. In future work, a camera module can be integrated into the STM32 system to capture images directly and perform real-time inference.

For future improvements, more greenhouse images should be collected, especially for rare and difficult classes such as tuta_leaf and weak pest symptoms. The dataset can also be expanded with images from different greenhouses, lighting conditions, camera angles, plant growth stages, and disease stages. In addition, stronger augmentation methods, improved preprocessing approaches, and different lightweight embedded models can be tested more systematically.

Overall, this project demonstrates a complete embedded AI workflow for tomato leaf health classification. The main contributions are the creation and cleaning of a custom real greenhouse dataset, the comparison of several model families and dataset versions, the selection of a practical front-side-only class structure, the use of an embedded-compatible ST FD-MobileNetV1 model, and the successful STM32 SD card based testing of the trained model.

REFERENCES

[1] M. H. Saleem, J. Potgieter and K. M. Arif, "Plant disease detection and classification by deep learning," Plants, vol. 8, no. 11, p. 468, 2019.

[2] K. Zhang, Q. Wu, A. Liu and X. Meng, "Can deep learning identify tomato leaf disease?," Advances in Multimedia, vol. 2018, Article ID 6710865, 2018.

[3] E. Özbilge, M. Köse Ulukök, Ö. Toygar and E. Özbilge, "Tomato disease recognition using a compact convolutional neural network," IEEE Access, vol. 10, pp. 77213-77224, 2022.

[4] H. Durmuş, E. O. Güneş and M. Kırcı, "Disease detection on the leaves of the tomato plants by using deep learning," in Proc. 2017 6th International Conference on Agro-Geoinformatics, 2017, pp. 1-5.

[5] S. Z. M. Zaki, M. A. Zulkifley, M. M. Stofa, N. A. M. Kamari and N. A. Mohamed, "Classification of tomato leaf diseases using MobileNet V2," IAES International Journal of Artificial Intelligence, vol. 9, no. 2, pp. 290-296, 2020.

[6] M. Umar, S. Altaf, S. Ahmad, H. Mahmoud, A. S. N. Mohamed and R. Ayub, "Precision agriculture through deep learning: Tomato plant multiple diseases recognition with CNN and improved YOLOv7," IEEE Access, vol. 12, pp. 49167-49183, 2024.

[7] H. Sun, R. Fu, X. Wang, Y. Wu, M. A. Al-Absi, Z. Cheng, Q. Chen and Y. Sun, "Efficient deep learning-based tomato leaf disease detection through global and local feature fusion," BMC Plant Biology, vol. 25, no. 311, 2025.

[8] B. Sowmya and S. Guruprasad, "Deep learning based plant health disease detection in tomatoes using Inception v4 convolutional neural network and YOLO V8," Discover Artificial Intelligence, vol. 5, no. 278, 2025.

[9] ISO/IEC/IEEE, "ISO/IEC/IEEE 12207:2026 Systems and software engineering — Software life cycle processes," International Organization for Standardization, 2026.

[10] ISO/IEC/IEEE, "ISO/IEC/IEEE 15288:2023 Systems and software engineering — System life cycle processes," International Organization for Standardization, 2023.

[11] ISO/IEC, "ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," International Organization for Standardization, 2023.

[12] ISO/IEC/IEEE, "ISO/IEC/IEEE 29119-1:2022 Software and systems engineering — Software testing — Part 1: General concepts," International Organization for Standardization, 2022.

[13] ISO/IEC, "ISO/IEC 22989:2022 Information technology — Artificial intelligence — Artificial intelligence concepts and terminology," International Organization for Standardization, 2022.

[14] ISO/IEC, "ISO/IEC 23053:2022 Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML)," International Organization for Standardization, 2022.

[15] ISO/IEC, "ISO/IEC 5338:2023 Information technology — Artificial intelligence — AI system life cycle processes," International Organization for Standardization, 2023.

[16] IEEE, "IEEE 1012-2024 Standard for System, Software, and Hardware Verification and Validation," IEEE Standards Association, 2024.

[17] ISO/IEC/IEEE, "ISO/IEC/IEEE 24748-7000:2022 Systems and software engineering — Life cycle management — Part 7000: Standard model process for addressing ethical concerns during system design," International Organization for Standardization, 2022.

APPENDICES

Appendix A

Final Training and Quantization Code

This appendix presents selected code blocks from the final ST FD-MobileNetV1 training notebook. The full notebook is not included in the main report in order to keep the document readable. The selected code blocks show the main settings used for dataset preparation, final class selection, training, quantization, software evaluation, and model saving.

A.1 General Settings

The final model was trained using five front-side tomato leaf classes. The input image size was selected as 224×224, and the dataset was divided into training, validation, and test subsets using a 70/15/15 split.

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

BASE_DRIVE = "/content/drive/MyDrive/bitirme_son3"
DATASET_SOURCE = os.path.join(BASE_DRIVE, "all_last7.zip")

OUT_DRIVE_DIR = os.path.join(
   BASE_DRIVE,
   "sadece_onler",
   "stm32_model_zoo_clean"
)

WORK_DIR = "/content/stm32_tomato_work"
RAW_DIR = os.path.join(WORK_DIR, "raw_dataset")
SPLIT_DIR = os.path.join(WORK_DIR, "split_dataset")

TRAIN_DIR = os.path.join(SPLIT_DIR, "train")
VAL_DIR   = os.path.join(SPLIT_DIR, "val")
TEST_DIR  = os.path.join(SPLIT_DIR, "test")

IMG_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 120

TRAIN_RATIO = 0.70
VAL_RATIO   = 0.15
TEST_RATIO  = 0.15
```

```
MIN_IMAGES_PER_CLASS = 20

KEEP_CLASSES = [
  "flies_leaf",
  "healthy_leaf",
  "nutdef_leaf",
  "trips_leaf",
  "tuta_leaf"
]

VALID_EXTS = (".jpg", ".jpeg", ".png", ".bmp", ".webp")
```

A.2 Dataset Preparation and Class Selection

The dataset folder names were already accepted as final class names. Therefore, the previous class merge block was removed from the final notebook. Only the selected five front-side classes were kept for the final model.

```
def list_images(folder):
  paths = []
  if not os.path.exists(folder):
    return paths

  for root, dirs, files in os.walk(folder):
    dirs[:] = [
      d for d in dirs
      if not d.startswith(".") and d != "__MACOSX"
    ]

    for f in files:
      if f.lower().endswith(VALID_EXTS):
        p = os.path.join(root, f)
        paths.append(p)

  return sorted(paths)
```

The final class order used during training, testing, TensorFlow Lite conversion, SD card export, and STM32 inference was:

```
class_names = [
  "flies_leaf",
  "healthy_leaf",
  "nutdef_leaf",
  "trips_leaf",
  "tuta_leaf"
]
```

A.3 Train / Validation / Test Split

The dataset was divided separately for each class in order to preserve the class structure in all subsets.

```
for cls in class_names:
   cls_dir = os.path.join(dataset_root, cls)

   paths = sorted([
      p for p in list_images(cls_dir)
      if p.lower().endswith(VALID_EXTS)
   ])

   train_paths, temp_paths = train_test_split(
      paths,
      train_size=TRAIN_RATIO,
      random_state=SEED,
      shuffle=True
   )

   val_relative = VAL_RATIO / (VAL_RATIO + TEST_RATIO)

   val_paths, test_paths = train_test_split(
      temp_paths,
      train_size=val_relative,
      random_state=SEED,
      shuffle=True
   )
```

The final dataset split was:

```
TRAIN
flies_leaf    : 1397
healthy_leaf  : 3027
nutdef_leaf   : 660
trips_leaf    : 1393
tuta_leaf     : 68
TOTAL         : 6545

VALIDATION
flies_leaf    : 300
healthy_leaf  : 649
nutdef_leaf   : 141
trips_leaf    : 299
tuta_leaf     : 15
TOTAL         : 1404
```

TEST
flies_leaf : 300
healthy_leaf : 649
nutdef_leaf : 142
trips_leaf : 299
tuta_leaf : 15
TOTAL : 1405

A.4 Final Model Configuration

The final embedded model was selected as ST FD-MobileNetV1 from the STM32 AI Model Zoo. The operation mode was selected as `chain_tqe`, which performs training, quantization, and quantized evaluation in one workflow.

```
general:
  project_name: tomato_stm32_st_fdmobilenetv1_5class_chain_tqe
  logs_dir: logs
  saved_models_dir: saved_models
  display_figures: False
  global_seed: 42
  gpu_memory_limit: 12

operation_mode: chain_tqe

model:
  model_name: st_fdmobilenetv1
  input_shape: (224, 224, 3)
  pretrained: False

dataset:
  dataset_name: custom_dataset
  class_names: [
    "flies_leaf",
    "healthy_leaf",
    "nutdef_leaf",
    "trips_leaf",
    "tuta_leaf"
  ]
  training_path: /content/stm32_tomato_work/split_dataset/train
  validation_path: /content/stm32_tomato_work/split_dataset/val
  test_path: /content/stm32_tomato_work/split_dataset/test
  quantization_split: 0.25
```

A.5 Preprocessing and Data Augmentation

The final model used 224×224 RGB input images. The images were normalized to the range expected by the ST model configuration. Moderate augmentation was used during training. Strong zooming was avoided because small disease and pest details could disappear when the image was cropped or zoomed too much.

```
preprocessing:
  rescaling:
    scale: 1/127.5
    offset: -1
  resizing:
    aspect_ratio: fit
    interpolation: area
  color_mode: rgb

data_augmentation:
  random_contrast:
    factor: 0.25
  random_brightness:
    factor: 0.12
  random_flip:
    mode: horizontal_and_vertical
  random_translation:
    width_factor: 0.04
    height_factor: 0.04
  random_rotation:
    factor: 0.06
  random_zoom:
    width_factor: 0.04
    height_factor: 0.04
```

A.6 Training and Quantization Settings

The model was trained with Adam optimizer. Learning rate reduction and early stopping were used to avoid unnecessary training after validation accuracy stopped improving. Post-training quantization was applied before STM32 deployment.

```
training:
  frozen_layers:
  dropout: 0.30
  batch_size: 32
  epochs: 120
  optimizer:
    Adam:
      learning_rate: 0.001
  callbacks:
    ReduceLROnPlateau:
      monitor: val_accuracy
      mode: max
      factor: 0.5
      patience: 8
      min_lr: 1.0e-6
    EarlyStopping:
```

```
    monitor: val_accuracy
    mode: max
    restore_best_weights: True
    patience: 25

quantization:
  quantizer: TFlite_converter
  quantization_type: PTQ
  quantization_input_type: uint8
  quantization_output_type: float
  export_dir: quantized_models
```

A.7 Software Test Evaluation

After training, the best saved model was loaded and evaluated on the test set. The test images were preprocessed using the same normalization as the ST model configuration.

```
model = tf.keras.models.load_model(latest_model, compile=False)

raw_test_ds = tf.keras.utils.image_dataset_from_directory(
    TEST_DIR,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    label_mode="categorical",
    shuffle=False
)

class_names = raw_test_ds.class_names

def st_preprocess(images, labels):
    images = tf.cast(images, tf.float32)
    images = images / 127.5 - 1.0
    return images, labels

test_ds = raw_test_ds.map(st_preprocess).prefetch(tf.data.AUTOTUNE)

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

test_loss, test_acc = model.evaluate(test_ds, verbose=1)
```

A.8 Classification Report and Confusion Matrix

The predicted class was selected using the maximum output probability. A classification report and confusion matrix were generated for the test set.

```
y_true = []
y_pred = []

for images, labels in test_ds:
   preds = model.predict(images, verbose=0)
   y_true.extend(np.argmax(labels.numpy(), axis=1))
   y_pred.extend(np.argmax(preds, axis=1))

report = classification_report(
   y_true,
   y_pred,
   target_names=class_names,
   digits=4,
   zero_division=0
)

cm = confusion_matrix(y_true, y_pred)

print(report)
print(cm)
```

The report, confusion matrix, and class list were saved to Google Drive for later comparison with the STM32 SD card test.

```
report_path = os.path.join(
   OUT_DRIVE_DIR,
   "st_fdmobilenetv1_test_report.txt"
)

cm_path = os.path.join(
   OUT_DRIVE_DIR,
   "st_fdmobilenetv1_confusion_matrix.csv"
)

classes_path = os.path.join(
   OUT_DRIVE_DIR,
   "st_fdmobilenetv1_classes.json"
)

with open(report_path, "w") as f:
   f.write("MODEL:\n")
   f.write(latest_model + "\n\n")
   f.write(f"TEST LOSS: {test_loss}\n")
```

```
    f.write(f"TEST ACC : {test_acc}\n\n")
    f.write("CLASSIFICATION REPORT:\n")
    f.write(report)
    f.write("\nCONFUSION MATRIX:\n")
    f.write(str(cm))

np.savetxt(cm_path, cm, delimiter=",", fmt="%d")

with open(classes_path, "w") as f:
    json.dump({i: c for i, c in enumerate(class_names)}, f, indent=2)
```

A.9 Final Model Saving

After training, the latest experiment folder, float model, quantized TensorFlow Lite model, test set, configuration file, and evaluation outputs were copied to Google Drive. This made the final experiment reproducible and preserved the exact model and test set used in the report.

```
FINAL_MODEL_DRIVE_DIR = os.path.join(
    OUT_DRIVE_DIR,
    "FINAL_MODEL_USED"
)

float_model_path = os.path.join(
    FINAL_MODEL_DRIVE_DIR,
    "final_float_model.keras"
)

quantized_model_path = os.path.join(
    FINAL_MODEL_DRIVE_DIR,
    "quantized_model.tflite"
)
```

The final saved files included:

```
final_float_model.keras
quantized_model.tflite
used_config_before_run.yaml
st_fdmobilenetv1_test_report.txt
st_fdmobilenetv1_confusion_matrix.csv
st_fdmobilenetv1_classes.json
TEST_SET_USED/
```

Appendix B

STM32 SD card UART output

Figure B.1 shows the final UART output obtained from the STM32 SD card test. In this test, the deployed ST FD-MobileNetV1 model was executed directly on the STM32F746G board. The test images were stored on the SD card as binary files and organized according to their class folders.

The STM32 board processed 1405 test images in total. Among these images, 1276 were classified correctly, giving a final embedded test accuracy of 90.81%. The failed directory and failed file counts were both zero, which shows that the SD card reading, file loading, input buffer preparation, and inference loop worked correctly during the test.

The UART output also includes the per-class results and the confusion matrix. The weakest class was flies, with 83.00% accuracy, while healthy, nutdef, trips, and tuta achieved higher per-class accuracies. The confusion matrix shows that most predictions were located on the diagonal, which confirms that the deployed model performed consistently on the STM32 board.

```
==================================
    SD TEST START
==================================

Expected BIN size: 150528 bytes
Expected folders : 00 to 04 under 0:/test
DIR OPEN OK: 0:/test/00 -> flies
DIR OPEN OK: 0:/test/01 -> healthy
DIR OPEN OK: 0:/test/02 -> nutdef
DIR OPEN OK: 0:/test/03 -> trips
DIR OPEN OK: 0:/test/04 -> tuta

==================================
    SD TEST FINISHED
==================================
```

Total files tested : 1405

Correct : 1276 / 1405

Accuracy : 90.81 %

Failed dirs : 0

Failed files : 0

Per-class results:

00 flies : 249 / 300 acc=83.00 %

01 healthy : 614 / 649 acc=94.60 %

02 nutdef : 133 / 142 acc=93.66 %

03 trips : 266 / 299 acc=88.96 %

04 tuta : 14 / 15 acc=93.33 %

Confusion matrix

Rows=true class, Cols=pred class

00 01 02 03 04

00: 249 39 6 6 0 | flies

01: 30 614 2 3 0 | healthy

02: 2 4 133 3 0 | nutdef

03: 16 12 4 266 1 | trips

04: 0 1 0 0 14 | tuta

Class index list:

00 -> flies

01 -> healthy

02 -> nutdef

03 -> trips

04 -> tuta

Test done.

Appendix C

STM32 AI Memory and Analyze Output

This appendix presents the memory analysis result of the final quantized ST FD-MobileNetV1 model. The analysis was performed using the STM32 AI tool for the STM32F7 target. The purpose of this analysis was to check whether the final quantized model was suitable for deployment on the STM32F746G board.

Table C.1 summarizes the most important memory and complexity values reported by the STM32 AI analyze tool.

Table C.1 STM32 AI analyze summary of the final quantized model

Parameter	Value
Model file	quantized_model.tflite
Target series	STM32F7
Model type	TensorFlow Lite
Input tensor	uint8, 1×224×224×3
Output tensor	float32, 1×5
Number of parameters	141,573
Weights	148,404 B / 144.93 KiB
Activations	211,840 B / 206.88 KiB
Input buffer	150,528 B / 147.00 KiB
Output buffer	20 B
Total Flash	207,531 B / 202.67 KiB
Total RAM	226,384 B / 221.08 KiB
MACC	approximately 24.74 million

The analyze result shows that the final quantized model was suitable for the STM32F746G target. The total Flash requirement was approximately 202.67 KiB, and the total RAM requirement was approximately 221.08 KiB. These values were within the memory limitations considered during the project.

The input buffer size was 150,528 bytes because the final model used 224×224 RGB input images with three channels. The output tensor had five values, corresponding to the five final front-side leaf classes.

This analysis supports the final model selection decision. EfficientNetB3 achieved higher software accuracy, but it was not suitable for the target embedded system. The final ST FD-MobileNetV1 model provided a better balance between accuracy, memory usage, and STM32 deployment feasibility.

Appendix D

Extended Technical Summary of Model Experiments

D.1 Main Model Experiment Results

Table D.1 summarizes the main numerical results obtained from different model groups.

Table D.1 Numerical summary of main model experiments

Model / Experiment	Classes	Input Size	Important Setting	Result	Final Use
Early compact CNN	7	192 / 224	Small custom CNN	76.50%	Not selected
ResNet-based baseline	6	224×224	High-capacity baseline	88.58%	Not selected
ST FD-MobileNet V1 early trial	9	224×224	STM32 AI Model Zoo model	86.82%	Not selected

ST FD-MobileNet V1 improved trial	7	224×224	Reduced class setup	87.40%	Not selected
SqueezeNet-inspired early trial	6	160×160	Custom lightweight CNN	82.20%	Not selected
SqueezeNet-inspired improved trial	6	160×160	Cleaner training flow	85.35% software / 85.61% quantized	Not selected
SqueezeNet-inspired larger input trial	6	192×192	Larger input resolution	85.61% software / 85.77% quantized	Not selected
Knowledge distillation trial	7	192×192	EfficientNetB3 teacher + SqueezeNet-inspired student	84.93% quantized	Not selected
Final SqueezeNet-inspired trial	5	192×192	Final front-side dataset	88.29% software / 88.36% quantized	Not selected
Early EfficientNetB3 trials	9	300×300	Wider dataset versions	about 87–89%	Upper-bound analysis
Reduced-class EfficientNetB3 trials	6	300×300	Class selection and augmentation trials	about 89–95%	Upper-bound analysis
Final EfficientNetB3 reference	5	300×300	Stronger training strategy	96.72%	Software reference
ST FD-MobileNet V1 five-class trial	5	224×224	Final dataset training trial	90.96% float / 91.10% quantized	Candidate

Final ST FD-MobileNet V1 software test	5	224×224	Final selected model	91.25%	Selected
Final ST FD-MobileNet V1 STM32 test	5	224×224	SD card embedded test	90.81%	Final embedded result

The results show that EfficientNetB3 achieved the highest software-level accuracy. However, it was not selected as the final model because the project required STM32 deployment. The SqueezeNet-inspired models were lightweight and quantization-friendly, but their accuracy remained lower than the final ST FD-MobileNetV1 model. ST FD-MobileNetV1 provided the best balance between accuracy and embedded deployment feasibility.

D.2 Final ST FD-MobileNetV1 Training Settings

Table D.2 gives the main training and preprocessing settings of the final ST FD-MobileNetV1 model.

Table D.2 Final ST FD-MobileNetV1 training configuration

Parameter	Value
Model	ST FD-MobileNetV1
Source	STM32 AI Model Zoo
Number of classes	5
Input shape	224×224×3
Dataset split	70% train, 15% validation, 15% test
Training images	6545
Validation images	1404
Test images	1405
Batch size	32
Maximum epochs	120
Optimizer	Adam
Initial learning rate	0.001
Dropout	0.30
Quantization type	Post-training quantization
Quantized input type	uint8
Quantized output type	float32

The final model used moderate augmentation. Strong zooming was avoided because small pest symptoms could disappear when the image was cropped or zoomed too much.

Table D.3 Final preprocessing and augmentation settings

Setting	Value
Rescaling	scale = 1/127.5, offset = -1
Color mode	RGB
Resize interpolation	Area
Random contrast	0.25
Random brightness	0.12
Random flip	Horizontal and vertical
Random translation	width = 0.04, height = 0.04
Random rotation	0.06
Random zoom	width = 0.04, height = 0.04

D.3 Final ST FD-MobileNetV1 Software Results

Table D.4 shows the software-level performance of the selected final model.

Table D.4 Final ST FD-MobileNetV1 software result

Metric	Value
Test images	1405
Test loss	0.2434
Test accuracy	91.25%
Macro F1-score	90.67%
Weighted F1-score	91.16%

Table D.5 shows the per-class software results.

Table D.5 Per-class software results

Class	Precision	Recall	F1-score	Support
flies_leaf	0.8836	0.8100	0.8452	300
healthy_leaf	0.9145	0.9553	0.9344	649
nutdef_leaf	0.9236	0.9366	0.9301	142
trips_leaf	0.9315	0.9097	0.9205	299
tuta_leaf	0.8750	0.9333	0.9032	15
Macro avg	0.9056	0.9090	0.9067	1405
Weighted avg	0.9120	0.9125	0.9116	1405

The weakest recall was obtained for flies_leaf. This class was difficult because fly evidence can be small and visually weak. The tuta_leaf class had a high F1-score, but its test support was only 15 images.

D.4 STM32 AI Analyze Results

The final quantized ST FD-MobileNetV1 model was analyzed using the STM32 AI tool for the STM32F7 target. Table D.7 summarizes the main memory and complexity values.

Table D.6 STM32 AI analyze summary

Parameter	Value
Model file	quantized_model.tflite
Target series	STM32F7
Number of parameters	141,573
Input tensor	uint8, 1×224×224×3
Output tensor	float32, 1×5
Weights	148,404 B / 144.93 KiB
Activations	211,840 B / 206.88 KiB
Input buffer	150,528 B / 147.00 KiB
Output buffer	20 B
Total Flash	207,531 B / 202.67 KiB
Total RAM	226,384 B / 221.08 KiB
MACC	approximately 24.74 million

These values show that the final quantized model was suitable for the STM32F746G target. The total Flash and RAM requirements were within the memory limitations considered during the project.

D.5 STM32 SD Card Test Results

The final quantized model was deployed on the STM32F746G board and tested using binary image files stored on the SD card.

Table D.7 Final STM32 SD card test result

Metric	Value
Total files tested	1405
Correct predictions	1276
STM32 accuracy	90.81%
Failed directories	0
Failed files	0
Input size	224×224×3
Expected binary file size	150,528 bytes
Number of output classes	5

Table D.8 Per-class STM32 results

Class	Correct / Total	Accuracy
flies_leaf	249 / 300	83.00%
healthy_leaf	614 / 649	94.60%
nutdef_leaf	133 / 142	93.66%
trips_leaf	266 / 299	88.96%
tuta_leaf	14 / 15	93.33%

The STM32 result was close to the software result. The software accuracy was 91.25%, and the embedded STM32 accuracy was 90.81%. This small difference showed that the model preserved most of its performance after conversion and deployment.

D.6 Final Selection Summary

The final model was selected by considering accuracy, memory usage, quantization behavior, and STM32 deployment feasibility together.

EfficientNetB3 achieved the highest software-level result with 96.72% accuracy. However, it was not selected as the final embedded model because it was larger and computationally heavier.

The SqueezeNet-inspired model was a useful custom lightweight candidate. Its best final result was 88.29% software accuracy and 88.36% quantized accuracy. However, this was lower than the final ST FD-MobileNetV1 result.

The final ST FD-MobileNetV1 model achieved 91.25% software accuracy and 90.81% STM32 embedded accuracy. It also required approximately 202.67 KiB Flash and 221.08 KiB RAM according to the STM32 AI analyze output. Therefore, ST FD-MobileNetV1 was selected as the final embedded model.

Appendix E

E.1 Preprocessing and STM32 Binary Export Code

This appendix presents the preprocessing and binary export code used to prepare test images for the STM32 SD card test. The final deployed model used 224×224 RGB input images. Therefore, each test image was resized to 224×224 pixels, converted to RGB format, and saved as a raw binary file.

The expected binary file size was:

224 × 224 × 3 = 150,528 bytes

This value was also checked in the STM32 code before inference. If the binary file size was different from the expected size, the file was not used for inference.

```
from PIL import Image
import numpy as np

IMG_SIZE = 224
```

```
def load_and_prepare_rgb_image(image_path):
    """
    Loads an image, converts it to RGB, resizes it to 224x224,
    and returns it as a uint8 NumPy array.
    """
    img = Image.open(image_path).convert("RGB")
    img = img.resize((IMG_SIZE, IMG_SIZE))
    arr = np.array(img, dtype=np.uint8)

    return arr
```

E.2 Binary Export Function

```
import os

EXPECTED_SIZE = IMG_SIZE * IMG_SIZE * 3

def save_image_as_bin(image_path, output_path):
    """
    Converts an RGB image into raw binary format for STM32 inference.
    The output file contains uint8 RGB pixel values.
    """
    arr = load_and_prepare_rgb_image(image_path)

    if arr.shape != (IMG_SIZE, IMG_SIZE, 3):
        raise ValueError(f"Unexpected image shape: {arr.shape}")

    arr.tofile(output_path)

    file_size = os.path.getsize(output_path)

    if file_size != EXPECTED_SIZE:
        raise ValueError(
            f"Wrong binary size. Expected {EXPECTED_SIZE}, got {file_size}"
        )
```

E.3 Exporting the Test Set

The test images were organized in class folders. The same class order was used during Python evaluation, TensorFlow Lite conversion, SD card export, and STM32 inference.

```
import os
from pathlib import Path

CLASS_NAMES = [
    "flies_leaf",
    "healthy_leaf",
```

```
    "nutdef_leaf",
    "trips_leaf",
    "tuta_leaf"
]

TEST_DIR = "/content/stm32_tomato_work/split_dataset/test"
BIN_OUTPUT_DIR = "/content/stm32_sd_test_bins"

os.makedirs(BIN_OUTPUT_DIR, exist_ok=True)

counter = 0

for class_index, class_name in enumerate(CLASS_NAMES):
    class_input_dir = os.path.join(TEST_DIR, class_name)
    class_output_dir = os.path.join(BIN_OUTPUT_DIR, f"{class_index:02d}_{class_name}")

    os.makedirs(class_output_dir, exist_ok=True)

    image_paths = sorted([
        str(p) for p in Path(class_input_dir).glob("*")
        if p.suffix.lower() in [".jpg", ".jpeg", ".png", ".bmp", ".webp"]
    ])

    for i, image_path in enumerate(image_paths):
        output_name = f"{i:06d}.BIN"
        output_path = os.path.join(class_output_dir, output_name)

        save_image_as_bin(image_path, output_path)
        counter += 1

print("Total exported binary files:", counter)
print("Expected file size:", EXPECTED_SIZE, "bytes")
```

E.4 SD Card Folder Structure

The exported binary files were copied to the microSD card using the following folder structure:

```
/test/
    00_flies_leaf/
        000000.BIN
        000001.BIN
        ...
    01_healthy_leaf/
        000000.BIN
        000001.BIN
        ...
```

```
02_nutdef_leaf/
   000000.BIN
   000001.BIN
   ...
03_trips_leaf/
   000000.BIN
   000001.BIN
   ...
04_tuta_leaf/
   000000.BIN
   000001.BIN
   ...
```

This folder structure allowed the STM32 code to compare the predicted class index with the true class folder. Therefore, the board could calculate total accuracy, per-class accuracy, and the confusion matrix during the SD card test.

E.5 Importance of Binary Export

The binary export step was necessary because the STM32 board did not load JPG or PNG images directly during the final test. Instead, the images were preprocessed on the computer and saved as raw RGB binary files. This reduced the complexity of the STM32 program and made the embedded test more repeatable.

Each binary file contained exactly 150,528 bytes. This value corresponds to a 224×224 RGB image with 3 channels. The STM32 code checked this size before inference to prevent invalid image files from being processed.